Fact checking con evidenza evidenziata — testa per token

Indice

Fact checking con evidenza evidenziata — testa per token **3**
1. Che cosa si vuole ottenere . . . 3
2. Perché “per token” . . . 4
3. Come funziona, passo per passo . . . 4
   3.1 Che cosa fa già il decision model . . . 4
   3.2 Che cosa aggiungiamo . . . 4
   3.3 Perché ogni pezzo . . . 5
   3.4 Perché il preambolo . . . 5
   3.5 Che cosa si addestra . . . 5
4. I modelli . . . 6
5. Glossario . . . 6
6. Sequenza di lavoro . . . 6
7. Obiettivi . . . 7

Passo 1 — I modelli e gli hook per token **8**
In breve . . . 8
1. Pacchetti . . . 8
2. L’architettura comune: Qwen3.5 . . . 8
3. Come si esegue ciascun modello . . . 8
4. Trovare i layer . . . 10
5. La riduzione dei vettori (PCA fissa) . . . 10
6. Hook sui vettori dei token . . . 10
7. Hook sull’attenzione . . . 11
8. Che cosa restituisce `evidence/models.py` . . . 11
9. Test . . . 11
10. GPU e spazio . . . 12

Passo 2 — I dati dalle fonti **12**
In breve . . . 12
1. Le fonti . . . 12
   SciFact . . . 13
   WiCE . . . 13
   HoVer con HotpotQA . . . 13
2. Preparazione . . . 14
3. Il comando . . . 14
4. Il formato di una riga oggi . . . 14
5. Il campo da aggiungere: `gold_char_sets` . . . 15
6. Controlli . . . 15

Passo 3 — Dati generati, etichette ricavate col modello, evidenza spostata **15**
In breve . . . 15
1. Documenti generati con posizioni note . . . 16
2. Etichette ricavate col modello (ablazione) . . . 16
3. Evidenza spostata . . . 17
4. Quante righe . . . 17

5. Controlli . . . 18

Passo 4 — Split e mix di training **18**
In breve . . . 18
1. Regole degli split . . . 18
2. I set . . . 18
3. Il mix `ev_tok_head` . . . 19
4. Comandi . . . 20

Passo 5 — Il prompt con il preambolo **20**
In breve . . . 20
1. Perché serve . . . 20
2. Esempio (KnowLine-4B, schema) . . . 20
3. Dove va il preambolo negli altri modelli . . . 21
4. Scegliere il testo del preambolo . . . 21
5. Riusare il verdetto o riaddestrarlo . . . 21
6. Dove si scrive la scelta . . . 22

Passo 6 — Dai caratteri ai token, e salvataggio dei vettori **22**
In breve . . . 22
1. L'esempio renderizzato . . . 22
2. Dai token ai caratteri del documento originale . . . 22
3. L'etichetta di un token . . . 24
4. Che cosa si salva (fase 1a) . . . 24
5. Comando . . . 24
6. Test . . . 24

Passo 7 — Metodi semplici da battere **25**
In breve . . . 25
1. I metodi . . . 25
2. Come si sceglie la combinazione di E-ATT . . . 26
3. Comandi . . . 26
4. Tabella da compilare (una per modello) . . . 26

Passo 8 — L'architettura della testa, la loss, la decodifica **26**
In breve . . . 26
1. La rete in una figura . . . 26
2. Gli ingressi . . . 27
3. Strato per strato . . . 27
   A. Miscela dei layer . . . 27
   B. Vettore del token . . . 27
   C. Contesto locale . . . 28
   D. Blocchi . . . 28
   E. Contesto globale . . . 28
   F–G. Ritorno ai token e punteggi . . . 28
4. Parametri e costo (misurati) . . . 28
5. Il codice della testa . . . 29
6. La loss . . . 30
   6.1 Il problema delle etichette . . . 30
   6.2 Le formule . . . 30
   6.3 Il codice . . . 30
   6.4 Esempio numerico (anche test automatico) . . . 30
7. La decodifica: dai punteggi ai tratti . . . 31
8. Test (`tests/test_token_head.py`) . . . 31

Passo 9 — Training **32**
In breve . . . 32
1. Fase 1a: il ciclo di training . . . 32

2. Scegliere i parametri della decodifica . . . 34
3. Che cosa guardare durante il training . . . 34
4. Fase 1b: vettori completi, riduzione appresa . . . 34
5. Fase 2: training anche del modello . . . 35
6. Test prima di ogni nuova configurazione . . . 35

Passo 10 — Valutazione **35**
In breve . . . 35
1. Metriche sulle frasi gold (le principali) . . . 36
2. Metriche sui caratteri . . . 36
3. Confronto con una testa per frase . . . 37
4. Prove di comportamento . . . 37
5. Latenza . . . 37
6. Comandi . . . 38
7. Report finale (uno per modello) . . . 38

Passo 11 — Esperimenti **38**
In breve . . . 38
Regole . . . 38
La testa di partenza: H0 . . . 39
Esperimenti principali (in quest'ordine) . . . 39
Controlli sulla configurazione migliore . . . 39
Testa per token contro testa per frase . . . 40
elephant-1 e gli altri modelli . . . 40
Controlli finali . . . 40
Registro . . . 40

Passo 12 — Server e rilascio **41**
In breve . . . 41
1. Che cosa fa il server per ogni richiesta . . . 41
2. Come gira ogni modello . . . 41
3. La cartella di rilascio . . . 42
4. Controlli prima del rilascio . . . 42
5. Congelamento, test, model card . . . 43

Checklist e problemi frequenti **43**
1. Checklist per passo . . . 43
2. Se succede... . . . 44

Una strada da esplorare: etichette per token dal modello stesso **44**
Il problema . . . 44
L'idea . . . 44
Come farlo . . . 45
Quando tenerlo . . . 45
Rischi . . . 45
Perché vale la pena . . . 45

Fact checking con evidenza evidenziata — testa per token

1. Che cosa si vuole ottenere

Abbiamo un **decision model** fine-tunato come fact checker: riceve un documento e un claim e risponde con P(supportato), in **un solo passaggio del modello sul testo** (un *forward*), senza generare parole. È veloce perché fa un solo forward.

Vogliamo che, quando il claim è supportato, dica anche **quali parti del documento lo dimostrano**, senza perdere velocità: sempre un solo forward, nessuna parola generata.

Esempio di risposta:

```
{"p_supported": 0.93, "supported": true,
 "evidence": [{"start": 0,    "end": 61,   "score": 0.91},
              {"start": 4210, "end": 4262, "score": 0.84}]}
```

- `start` / `end`: inizio e fine, in caratteri, di un tratto di testo del documento originale (`document[start:end]`).
- Se il claim non è supportato, `evidence` è vuota: non si indica né cosa manca né cosa lo contraddice.

Il caso difficile è l'**evidenza sparsa**: due parti lontane che solo **insieme** dimostrano il claim.

> Claim: "Nancy R. Howell teaches at a seminary in Kansas City." Inizio del documento: "Saint Paul School of Theology is a seminary located in Kansas City, Missouri." … 38 frasi che non c'entrano … Fine del documento: "Nancy R. Howell is Professor of Theology at Saint Paul School of Theology."
>
> Nessuna delle due basta da sola. Evidenza corretta: entrambe.

2. Perché "per token"

La testa dà un punteggio a **ogni token** del documento, e le parti mostrate sono i tratti di token sopra soglia. Non c'è nessuna divisione del documento in frasi, né in training né in produzione. Quindi:

- il risultato **non dipende da come il testo è diviso in frasi** (nessun NLTK, nessuna regola di punteggiatura da mantenere identica fra training e produzione);
- funziona anche su testo senza punteggiatura, elenchi, tabelle scritte come testo;
- può evidenziare **solo la parte utile** di una frase lunga.

Se all'utente si vogliono mostrare frasi intere, i tratti si allargano alla frase che li contiene **solo in visualizzazione**, con qualunque divisore: la testa non ne dipende.

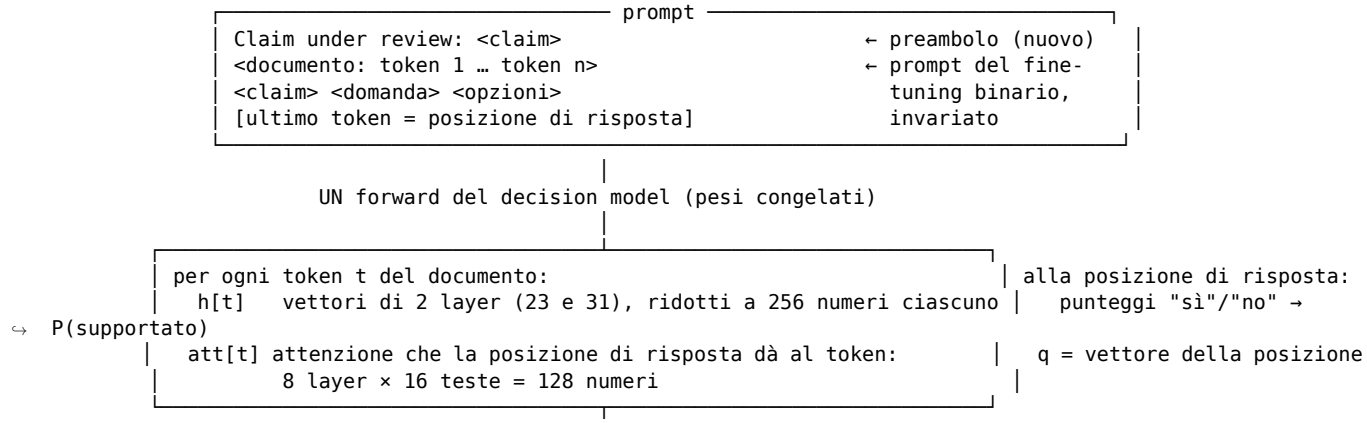
3. Come funziona, passo per passo

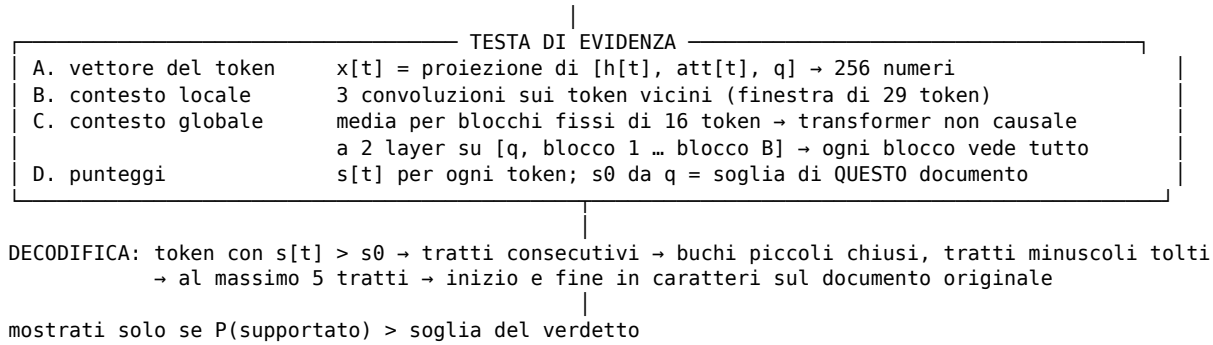
3.1 Che cosa fa già il decision model

Il modello legge il prompt (documento, claim, domanda). Per **ogni token** e per **ogni layer** calcola un vettore di numeri (lo *stato nascosto*): 2.560 numeri nei modelli da 4B, 4.096 in quelli da 9B. Il verdetto si legge dall'**ultimo token del prompt** (la *posizione di risposta*): lì il modello dà un punteggio alla risposta "sì" e uno alla risposta "no", da cui P(supportato). Questa parte esiste già: è il modello del fine-tuning binario, addestrato con una LoRA sulla domanda di fact checking. **Non la tocchiamo.**

3.2 Che cosa aggiungiamo

Durante quel forward leggiamo alcuni dei vettori che il modello calcola comunque, e li diamo a una **piccola rete separata**, la *testa di evidenza* (4,5 milioni di parametri contro i 4–9 miliardi del modello), che dà un punteggio a ogni token del documento.





L'architettura completa, con forme dei tensori, parametri e costo di ogni parte, è nel passo 08.

3.3 Perché ogni pezzo

Pezzo	Perché serve
vettori da 2 layer, non solo dall'ultimo	l'ultimo layer è specializzato a predire la risposta; un layer intermedio rappresenta meglio il contenuto dei token
attenzione della posizione di risposta	dice su quali token il modello si è basato per decidere: un indizio forte, già calcolato
`q` sommato a ogni token	dice alla testa qual è il claim e cosa ha deciso il modello
contesto locale	un token da solo non basta: una parola è evidenza perché sta in una certa frase
contesto globale a blocchi	l'evidenza sparsa richiede che parti lontane si vedano; un'attenzione piena su migliaia di token costa troppo, su ~100–400 blocchi è trascurabile. I blocchi sono fissi (16 token per posizione): nessuna dipendenza dal significato o dalla punteggiatura
soglia `s0` del documento	non c'è una soglia unica da tarare; in un caso multi-hop la soglia può scendere per far passare la parte "ponte"

3.4 Perché il preambolo

Nei modelli **causali** (come KnowLine-4B) ogni token vede solo ciò che lo precede. Nel prompt del fine-tuning binario il claim viene **dopo** il documento: quando il modello legge il documento non sa ancora cosa deve verificare, e i vettori dei token del documento non "parlano" del claim. Mettendo una riga `Claim under review: <claim>` **prima** del documento, ogni token è letto sapendo cosa si verifica. Il resto del prompt resta identico, compreso il claim dopo il documento dove il verdetto è abituato a vederlo. Dettagli nel passo 05.

3.5 Che cosa si addestra

Solo la testa. Il modello resta congelato, quindi **il verdetto non cambia**. Le etichette sono le frasi di evidenza annotate nei dataset (SciFact, WiCE, HoVer), trasformate in intervalli di caratteri e poi in token: un token è "evidenza" se sta dentro una di quelle frasi. La loss è pensata per il fatto che le annotazioni sono a livello di frase: chiede che ogni frase gold sia colpita e che nulla si accenda fuori, non che ogni token della frase sia acceso.

Siccome il modello è congelato, i suoi vettori si calcolano **una volta** e si salvano su disco (ridotti a 256 numeri per token); poi ogni prova di training della testa dura minuti.

4. I modelli

I 10 decision model dello studio focalizzato, tutti costruiti su **Qwen3.5**: 32 layer, di cui 8 ad attenzione piena (layer 3, 7, 11, 15, 19, 23, 27, 31) e 24 Gated DeltaNet (attenzione lineare, sempre causale).

Modello	Ruolo	d	Attenzione nei layer pieni
KnowLine-4B (`PelaAI/KnowLine-4B-Gen5.5`)	tutte le prove si fanno qui	2.560	causale
elephant-1 (`baselabs/elephant-1`)	conferma a 9B	4.096	bidirezionale a blocchi
Jeffan-4B, d3-flash, OneJev1.6-9B, Nimble-9B-v3	estensione	2.560 / 4.096	Jeffan, d3: bidirezionale; OneJev, Nimble: causale
StartLux 9B/4B, pplx-decider-flash, Clef-Flash	estensione, più lavoro	2.560 / 4.096	StartLux, pplx: bidirezionale; Clef: come caricato

Ogni modello si esegue **con il codice dei suoi autori** (prompt, maschera di attenzione, lettura della risposta), come nei benchmark della repo. Dettagli nel passo 01.

5. Glossario

Termine	Significato
forward	un passaggio del modello su tutto il prompt
stato nascosto, vettore	i numeri che il modello calcola per un token in un layer
posizione di risposta	il token del prompt dove si legge il verdetto (l'ultimo)
token del documento	i token del prompt che appartengono al documento
testa di evidenza	la piccola rete che dà un punteggio a ogni token del documento
`s[t]`, `s0`	punteggio del token t; soglia del documento
blocco	16 token consecutivi del documento: unità interna della testa per il contesto globale
tratto	sequenza di token consecutivi mostrati come evidenza, convertita in `[inizio, fine)` di caratteri
frase gold	frase di evidenza annotata nel dataset, come intervallo di caratteri
colpita	una frase gold è colpita se almeno metà dei suoi caratteri è dentro i tratti mostrati
insieme completo	tutte le frasi gold di una riga colpite
sufficienza	il documento ridotto ai soli tratti mostrati fa ancora dire "supportato" al modello
fase 1a / 1b / 2	training della testa su vettori salvati / con il modello in memoria / insieme a una LoRA del modello

6. Sequenza di lavoro

#	Passo	File	Prodotto
1	Modelli e hook per token	01	forward con cattura dei vettori e dell'attenzione per token
2	Dati dalle fonti	02	`dataset/evidence/*.jsonl` con le frasi gold in caratteri
3	Dati generati (facoltativo per le prime prove)	03	evidenza esatta, etichette dal modello, evidenza spostata
4	Split e mix	04	set di valutazione congelati, mix di training
5	Prompt con il preambolo	05	preambolo scelto, verdetto verificato
6	Dai caratteri ai token, estrazione	06	etichette per token, vettori su disco
7	Baseline	07	numeri da battere
8	Architettura, loss, decodifica	08	`evidence/token_head.py`, `token_losses.py`, `decode.py`
9	Training	09	teste addestrate
10	Valutazione	10	metriche e latenza
11	Esperimenti	11	configurazione finale
12	Server e rilascio	12	server che restituisce verdetto e tratti
—	Checklist	13	
—	Una strada da esplorare	14	

Codice già pronto nella repo: `evidence/data/` (costruzione dei dataset dal passo 2) e `tests/test_evidence_data.`
Il resto è da scrivere, con i file indicati in ogni passo.

7. Obiettivi

Obiettivo	Soglia
Verdetto	uguale a quello del fine-tuning binario (BAcc e AUC entro 0,3 punti)
Frasi gold colpite	≥ 0,80 su HoVer
Insieme completo su HoVer	≥ 0,65
F1 sui caratteri	almeno 15 punti sopra la baseline lessicale (E-LEX, 07)
Evidenza lontana	frasi colpite con evidenza a ≥ 20 frasi di distanza al massimo 5 punti sotto quelle vicine
Documento senza punteggiatura	frasi colpite al massimo 5 punti sotto il documento normale
Sufficienza	≥ 85%
Tratti	in media al massimo 5 per riga; nessun tratto sotto 4 token
Latenza	al massimo +3% del solo verdetto (modelli 4B: +3% o +3 ms)

Passo 1 — I modelli e gli hook per token

In breve

Serve un modulo, `evidence/models.py`, che per ciascun modello faccia **un forward** con il codice dei suoi autori e restituisca, dallo stesso forward:

1. i punteggi delle due opzioni nella posizione di risposta → P(supportato), come nel fine-tuning binario;
2. i vettori di **tutti i token del documento** in 2 layer, ridotti subito a 256 numeri per layer (§5);
3. per gli 8 layer ad attenzione piena, quanto la posizione di risposta "guarda" **ogni token** (una riga di pesi di attenzione per testa);
4. il vettore della posizione di risposta (`q`).

I punti 2–4 si ottengono con degli *hook*: piccole funzioni agganciate ai layer, che copiano i dati mentre il forward passa, senza cambiarlo.

1. Pacchetti

Ambiente `factcheck` della repo (transformers 5.14, torch, accelerate, peft), più:

```
pip install flash-linear-attention causal-conv1d safetensors rank_bm25 sentence-transformers
```

`flash-linear-attention` e `causal-conv1d` servono ai 24 layer Gated DeltaNet di Qwen3.5: senza, transformers usa un'implementazione lenta in torch. Controllo: `python -c "import fla, causal_conv1d"`.

2. L'architettura comune: Qwen3.5

Tutti i modelli sono Qwen3.5 con la stessa struttura:

	Gruppo 4B	Gruppo 9B
Layer	32	32
Layer ad attenzione piena	3, 7, 11, 15, 19, 23, 27, 31	idem
Altri layer	24 Gated DeltaNet: portano avanti un riassunto del testo letto, sempre causali	idem
Teste di query / di chiavi (layer pieni)	16 / 4	16 / 4
Lunghezza del vettore (d)	2.560	4.096
Tokenizer	Qwen3.5 (comune a tutti)	idem

Layer da cui si leggono i vettori dei token: **23 e 31** (penultimo e ultimo layer pieno; il 31 è anche l'ultimo layer). L'esperimento H6 (11) prova 19, 23, 27, 31.

3. Come si esegue ciascun modello

Ogni modello si esegue con il codice dei suoi autori: prompt, maschera di attenzione e lettura della risposta sono i loro. Caricare i pesi con transformers "normale" darebbe, per i modelli bidirezionali, un modello causale che risponde diversamente **senza dare errori**.

Modello	Codice degli autori	Prompt (schema)	Attenzione	Lettura della risposta	Posizione di risposta
KnowLine-4B	front end nel repository del modello (`knowline_server.py`, `knowline_layout.py`), backend transformers	stato come testo indentato in `key: value`, un turno per riga; turno finale con istruzione, `Question:`, `A. Yes: …` / `B. No: …`; risposta aperta con `Answer:`. Richieste fino a 3.000 token **scritte due volte**	causale	token A/B; temperatura 1,50 già nei pesi	ultimo token
elephant-1	motore `qev` di OneJev (copia in `models/elephant-1/qev`)	sistema + `<state>` JSON indentato `</state>` come prefisso; la domanda come suffisso con `A. yes` / `B. no`	bidirezionale a blocchi nei layer pieni (prefisso con sé stesso, suffisso con prefisso e sé stesso), dichiarata in `modeling_overrides` del config	lettere A/B	ultimo token del suffisso
Jeffan-4B	`inference/jeffan_engine` (nel repository)	`State:` JSON, `Question:`, `A: No / false`, `B: Yes / true`	bidirezionale nei layer pieni	codici A/B	ultimo token
d3-flash	codice del modello (`trust_remote_code`)	`State:` JSON, `Question:`, `A: No / false`, `B: Yes / true`	non causale nei layer pieni	codici fra 255	ultimo token
OneJev1.6-9B	renderer di Flide; in transformers come sequenza unica	prefisso (sistema + stato) e suffisso della domanda	causale	lettere A/B	ultimo token
Nimble-9B-v3	motore degli autori (`models/nimble-9b/nimble`), LoRA su Qwen3.5-9B	turno JSON con contesto e schema, `Requested field: "<id>"`	causale	codici fra 255	ultimo token
StartLux 9B/4B	pacchetto `startlux-decide` (gli autori usano vLLM con un plugin; in transformers la maschera va ricostruita)	`Evidence:` JSON compatto, `Question:`, `A) yes` / `B) no`	bidirezionale sul prompt	lettere A/B, temperatura per tipo	ultimo token

Modello	Codice degli autori	Prompt (schema)	Attenzione	Lettura della risposta	Posizione di risposta
pplx-decider-flash	pacchetto `pplx_decider` (percorso di riferimento)	domande in una sequenza, ognuna letta nella sua posizione	non causale nei layer pieni	testa propria a 255 vie	posizione di lettura della domanda
Clef-Flash	`joint_schema_model.py`	STATE: JSON, SCHEMA FIELDS:	come caricato	testa congiunta propria	ultimo token prima della testa

4. Trovare i layer

```python
import torch

def text_layers(model, n_layers=32):
    """La ModuleList dei layer testuali (i modelli caricati come multimodali hanno anche una torre visiva)."""
    found = [m for name, m in model.named_modules()
             if isinstance(m, torch.nn.ModuleList) and len(m) == n_layers and "visual" not in name and "vision"
              ↪  not in name]
    assert len(found) == 1
    return found[0]

def full_attention_layers(config):
    tc = getattr(config, "text_config", config)
    return [i for i, t in enumerate(tc.layer_types) if t == "full_attention"]     # [3, 7, 11, 15, 19, 23, 27, 31]
```

5. La riduzione dei vettori (PCA fissa)

I vettori dei token sono tanti (~1.500 per un documento HoVer, fino a ~18.000 per WiCE) e lunghi (2.560 numeri). Salvarli così occuperebbe terabyte. Si riducono a 256 numeri per layer con una **PCA calcolata una volta** per modello e layer:

```python
def fit_pca(samples, k=256):
    """samples: [M, D] vettori di token presi a caso. -> (media [D], proiezione [D, k], quota di varianza
     ↪  spiegata)."""
    mean = samples.mean(0)
    x = samples - mean
    U, S, V = torch.pca_lowrank(x, q=k, center=False)
    explained = (S ** 2).sum() / (x ** 2).sum()
    return mean, V[:, :k], float(explained)
```

1. 2.000 righe a caso del mix di training; per ognuna, 200 token a caso del documento; 400.000 vettori per layer.
2. `fit_pca` per i layer 23 e 31.
3. Salvataggio in `data/evidence_tok/pca/<modello>/layer<i>.safetensors` (media e proiezione) con la varianza spiegata. Se nel layer 31 è sotto il 70%, si passa a 384 componenti.

La PCA serve solo a salvare i vettori per la fase 1a. Nella fase 1b, con il modello in memoria, la testa impara la propria proiezione partendo dalla PCA (09 §4).

6. Hook sui vettori dei token

```python
class TokenTap:
    """Per ogni layer scelto: i vettori dei token del documento, proiettati a 256, e il vettore della posizione
     ↪  di risposta."""
    def __init__(self, layers, layer_idx, pca):
        self.out, self.doc, self.answer_pos = {}, None, None         # doc = (primo, ultimo) token del documento
         ↪  nel prompt
        self.pca = pca                                                # {layer: (media [D], proiezione [D, 256])}
         ↪  sulla GPU
        self.handles = [layers[i].register_forward_hook(self._hook(i)) for i in layer_idx]

    def _hook(self, i):
```

```
        def hook(module, args, output):
            h = (output[0] if isinstance(output, tuple) else output)[0]          # [T, D]
            a, b = self.doc
            mean, proj = self.pca[i]
            self.out[i] = {"tok": ((h[a:b + 1].float() - mean) @ proj).half(),  # [n, 256]
                           "q": h[self.answer_pos].float()}                      # [D]
        return hook
```

La proiezione si fa sulla GPU dentro l'hook: dalla GPU escono 256 numeri per token e per layer. Non si conservano mai i vettori completi di tutti i token.

7. Hook sull'attenzione

Le implementazioni veloci dell'attenzione non restituiscono i pesi, e chiederli in modalità "eager" costa troppa memoria con documenti lunghi. Si registra una funzione di attenzione che calcola l'attenzione normale e, negli 8 layer pieni, ricalcola **una riga** di pesi: quella della posizione di risposta verso tutti i token. Costa pochissimo.

```
from transformers import AttentionInterface
from transformers.integrations.sdpa_attention import sdpa_attention_forward

CAPTURE = {"rows": {}, "full": {3, 7, 11, 15, 19, 23, 27, 31}, "query_pos": -1, "inner": sdpa_attention_forward}

def sdpa_capture(module, query, key, value, attention_mask, scaling=None, **kw):
    out = CAPTURE["inner"](module, query, key, value, attention_mask, scaling=scaling, **kw)
    if module.layer_idx in CAPTURE["full"]:
        qp = CAPTURE["query_pos"]
        sl = slice(qp, qp + 1 if qp != -1 else None)
        k = key.repeat_interleave(query.size(1) // key.size(1), dim=1)          # 16 teste di query, 4 di chiavi
        s = (query[:, :, sl] @ k.transpose(-1, -2)).float() * scaling           # [B, 16, 1, T]
        if attention_mask is not None:                                          # la maschera del modello, anche
         ↪  bidirezionale
            s = s + attention_mask[:, :, sl, : s.size(-1)].float()
        CAPTURE["rows"][module.layer_idx] = s.softmax(-1)[:, :, 0]              # [B, 16, T]
    return out

AttentionInterface.register("sdpa_capture", sdpa_capture)
# caricamento: from_pretrained(..., attn_implementation="sdpa_capture")
```

- In transformers 5.14 la classe di attenzione di Qwen3.5 passa per `AttentionInterface` e ha `layer_idx`; `query` e `key` arrivano già pronte (dopo le rotazioni di posizione), `scaling` arriva dal modello.
- Se il codice degli autori installa una propria funzione di attenzione per la maschera bidirezionale (pplx, Jeffan, d3-flash, elephant-1), la si mette in `CAPTURE["inner"]` al posto di `sdpa_attention_forward`: la maschera resta la loro.

Dalla riga di pesi alla feature per token: `att[t] = [riga del layer l, testa j, posizione t]` per gli 8 layer e le 16 teste, cioè 128 numeri per token del documento.

8. Che cosa restituisce `evidence/models.py`

```
def forward_tokens(model, rendered):
    """rendered: input_ids, answer_pos, doc_tok (primo, ultimo token del documento) -> dict:
       answer_logits [2]  punteggi delle opzioni sì/no (per P(supportato))
       h   [n, 2, 256]    vettori dei token del documento, layer 23 e 31, proiettati
       att [n, 128]       attenzione della posizione di risposta sui token del documento
       q   [2, D]         vettore della posizione di risposta, layer 23 e 31"""
```

9. Test

`python -m evidence.models --model knowline-4b --selftest`, poi per gli altri modelli:

Test	Come	Deve dare
Verdetto invariato	50 righe di dev, prompt senza preambolo: P(supportato) dal forward con gli hook contro l'adapter del benchmark	differenza media < 0,005, massima < 0,02
Layer giusti	vettore dell'hook al layer i (prima della PCA) = `hidden_states[i + 1]` di transformers	identico
Attenzione giusta	5 prompt da ~1.000 token, contro la modalità "eager" con la stessa maschera	differenza < 0,001
Maschera conservata	per i modelli bidirezionali, il verdetto con e senza hook	identico
Documento giusto	i token `doc_tok` decodificati sono il documento (a meno dell'escaping del prompt)	200 righe su 200
PCA	proiezione nell'hook = proiezione dopo, sugli stati completi	differenza < 1e-2 (fp16)

10. GPU e spazio

Uso	Gruppo 4B	Gruppo 9B
Estrazione dei vettori	24 GB	45 GB per documenti lunghi
Training con il modello in memoria (fase 1b)	24 GB	45 GB
Training anche del modello (fase 2)	24 GB	45 GB

Spazio su disco per riga, 2 layer: 1 KB per token di `h` + 256 B di `att`. Per le ~28.600 righe di SciFact, WiCE e HoVer: circa 60 GB.

Passo 2 — I dati dalle fonti

In breve

Per addestrare e valutare la testa servono righe con documento, claim, etichetta e, per i positivi, **dove sono le frasi di evidenza** nel documento, in caratteri. Si usano tre dataset pubblici che hanno già l'evidenza annotata: **SciFact**, **WiCE**, **HoVer** (con HotpotQA distractor per il testo delle pagine).

La costruzione è già scritta, in `evidence/data/` (comando `python -m evidence.data.build`). Per la testa per token va aggiunto un campo: le frasi gold come intervalli di caratteri (§5).

1. Le fonti

Fonte	Da dove	Righe	Positivi / negativi	Documento
SciFact	`allenai/scifact_entailment` (cache HF), train + validation	1.258	508 / 750	titolo + abstract biomedico (mediana 9 frasi, ~590 token di prompt)

Fonte	Da dove	Righe	Positivi / negativi	Documento
WiCE	`tasksource/wice` (cache HF), train + dev + test	5.294	3.100 / 2.194	la pagina web citata da Wikipedia, una riga per elemento (mediana 68 frasi, ~1.670 token, massimo 18.580)
HoVer	claim da `dataset/HoVer/data/hover_{train,dev}_release_v1.1.json`; testo da `hotpotqa/hotpot_qa` (distractor)	22.063	12.974 / 9.089	~10 pagine Wikipedia: quelle dell'evidenza più pagine di disturbo (mediana 46 frasi, ~1.690 token)

SciFact

- `SUPPORT` → positivo; le frasi di *rationale* sono l'evidenza.
- `CONTRADICT` → negativo; le frasi che contraddicono si salvano a parte (`contra`), solo per analisi.
- `NEI` → negativo.
- Documento: titolo, a capo, frasi dell'abstract unite da spazio.

WiCE

- `supported` → positivo; `supporting_sentences` dà **più insiemi alternativi** di frasi che dimostrano il claim.
- `not_supported` → negativo.
- `partially_supported` → etichetta 0 ma **nessuna loss di evidenza** (una parte del claim è supportata, quindi non si può chiedere che nessuna frase si accenda), oppure scartato con `--wice-partial drop`.
- Documento: gli elementi della pagina (titoli, righe, frasi) uniti da a capo.
- **WiCE è anche una fonte del test LLM-AggreFact.** Ogni riga ha `meta.aggrefact_overlap`: `"dev"` o `"test"` se la stessa pagina è in AggreFact dev o test, `false` altrimenti. Il confronto si fa sulla pagina, non sul claim, perché AggreFact usa i claim interi e WiCE li divide in pezzi. Misurato: tutto il dev di WiCE è in AggreFact dev, tutto il test di WiCE in AggreFact test, più 2 righe del train. Queste righe non vanno mai in training (04).

HoVer con HotpotQA

- HoVer dà i claim (2–4 "salti" fra pagine), le etichette `SUPPORTED` / `NOT_SUPPORTED` e le frasi di evidenza come (titolo della pagina, numero di frase), con la numerazione di HotpotQA.
- HotpotQA distractor, collegato con `hpqa_id`, dà per quella domanda le pagine già divise in frasi e ~8 pagine di disturbo. Il documento contiene le pagine dell'evidenza e quelle di disturbo, in ordine casuale fissato dall'id del claim: ogni pagina è titolo, a capo, testo; pagine separate da una riga vuota. L'evidenza è quindi spesso lontana: in 6.608 positivi su 12.974 le frasi gold distano 20 o più frasi.
- Per i claim a 3–4 salti, HotpotQA non ha tutte le pagine. Con `--hover-db-fallback` le pagine mancanti si prendono dal database Wikipedia di HoVer (`dataset/HoVer/data/wiki_wo_links.db`), e la frase n si trova dividendo il testo con il nostro divisore. Senza questa opzione si perde il 45% dei claim. Misurato su 5.488 frasi di pagine presenti in entrambe le fonti: la frase così

trovata è quella giusta nel 98% dei casi; nel 2% è sfasata di una frase. Queste righe hanno `meta.gold_alignment = "db_split"`, le altre `"exact"`.

Il divisore in frasi entra **solo qui**, per creare le etichette delle pagine prese dal db: mai nel modello.

2. Preparazione

```
python -c "import nltk; nltk.download('punkt_tab')"        # divisore usato dal build (solo per i dati)
chmod -R go-w ~/nltk_data                                  # NLTK rifiuta una cartella dati scrivibile da altri
↪  utenti
hf download hotpotqa/hotpot_qa --repo-type dataset --include "distractor/*"
```

SciFact e WiCE devono essere nella cache HF; HoVer in `dataset/HoVer/`; LLM-AggreFact in `dataset/lytang--LLM-AggreFact/` (per il controllo su WiCE); il tokenizer di KnowLine-4B nella cache HF (per il filtro di lunghezza).

3. Il comando

```
python -m evidence.data.build --source scifact wice hover --hover-db-fallback
```

Opzione	Default	Cosa fa
`--hover-db-fallback`	spento	pagine HoVer mancanti prese dal db (da tenere acceso)
`--max-tokens N`	32.768	scarta le righe il cui prompt supera N token del tokenizer di KnowLine-4B: documento + 2 × claim + 256 (il front end di KnowLine aggiunge 157 token, misurati)
`--wice-partial`	`negative`	WiCE parzialmente supportati: etichetta 0 senza loss di evidenza, o `drop`
`--limit N`	0	primi N record per fonte, per prove

Uscita in `dataset/evidence/`: `<fonte>.jsonl`, `<fonte>.stats.json` (conteggi, scarti con motivo, lunghezze, distanze fra frasi gold), `manifest.json` (file letti, revisioni, sha256, opzioni, commit). Circa un minuto e mezzo.

Con il limite di default non si scarta nessuna riga per lunghezza. Con `--max-tokens 8192` si scartano 75 righe di WiCE; con 4.096, 503 di WiCE e 2 di HoVer.

4. Il formato di una riga oggi


```
{"id": "hover:train:330ca632-…", "source": "hover", "orig_split": "train",
 "claim": "…", "document": "Kristian Zahrtmann\nKristian Zahrtmann was …\n\nOssian Elgström\n…",
 "label": 1,
 "sentences": [[0, 61], [62, 140], "…"],
 "evidence": [1, 9, 31], "evidence_sets": [[1, 9, 31]],
 "contra": [],
 "evidence_quality": "gold", "evidence_term": "supervised",
 "doc_id": "hover:5ab7a86d5542995dae37e986",
 "meta": {"orig_label": "SUPPORTED", "gold_alignment": "exact", "doc_tokens": 1534, "prompt_tokens": 1830, "…":
 ↪  "…"}}
```


Campo	Significato
`label`	1 = supportato, 0 = no

Campo	Significato
`sentences`, `evidence`, `evidence_sets`	frasi del nostro divisore e indici di quelle di evidenza: servono solo alla valutazione per frase (10 §3), la testa per token non li usa
`evidence_term`	che loss riceve la riga: `supervised` (positivo con evidenza), `mil_neg` (negativo: nessun token deve accendersi), `none` (nessuna loss di evidenza)
`doc_id`	righe con lo stesso documento (lo split si fa per `doc_id`)
`meta.prompt_tokens`	lunghezza stimata del prompt in token

5. Il campo da aggiungere: `gold_char_sets`

La testa per token impara dalle frasi gold **come le dà la fonte**, in caratteri, senza passare dal nostro divisore. Il builder le calcola già (servono a ricavare `evidence`), ma non le salva. Si aggiungono due campi:

```
"gold_char_sets": [[[412, 497], [3120, 3188]]],
"contra_char": []
```

- `gold_char_sets`: per ogni insieme alternativo di evidenza, gli intervalli `[inizio, fine)` di caratteri delle frasi gold (frasi di SciFact, elementi di WiCE, frasi HotpotQA di HoVer; per le pagine HoVer prese dal db, le frasi trovate dividendo il testo);
- `contra_char`: lo stesso per le frasi che contraddicono (SciFact).

Modifica in `evidence/data/schema.py`, funzione `assemble`:

```
    row["gold_char_sets"] = [[list(span) for span in gold] for gold in gold_sets if gold]
    row["contra_char"] = [list(span) for span in contra]
```

e in `validate`:

```
    for spans in row["gold_char_sets"] + [row["contra_char"]]:
        for s, e in spans:
            if not (0 <= s < e <= len(row["document"])) or not row["document"][s:e].strip():
                raise ValueError(f"{row['id']}: bad gold character span {(s, e)}")
    if row["label"] == 0 and row["gold_char_sets"]:
        raise ValueError(f"{row['id']}: label 0 with gold spans")
```

Poi un test in `tests/test_evidence_data.py` (il testo di ogni intervallo è la frase gold della fonte) e la ricostruzione dei dataset.

6. Controlli

Controllo	Atteso
Ogni intervallo di `gold_char_sets` dentro il documento, non vuoto	tutte le righe
Testo degli intervalli = frase gold della fonte (spazi ai bordi a parte)	tutte le righe `exact`
30 positivi per fonte letti a mano con gli intervalli evidenziati (`python -m evidence.data.show`)	le parti evidenziate dimostrano il claim in almeno 28 su 30

Passo 3 — Dati generati, etichette ricavate col modello, evidenza spostata

In breve

Le tre fonti del passo 2 hanno due limiti: l'evidenza annotata è a livello di frase intera, e i testi sono quasi solo Wikipedia, pagine web e abstract biomedici. Tre famiglie di dati in più li correggono. Per

le prime prove si possono saltare; servono quando la testa funziona e si vuole migliorarla.

Famiglia	Idea	Che cosa porta	Script
Generati con posizioni note	un LLM scrive un articolo che contiene fatti dati; si sa esattamente dove sono	etichette **esatte**, anche dentro una frase; testi tipo notizia	`evidence/data/gen_marked.py`
Etichettati col modello	per documenti senza evidenza annotata, si cerca col modello il gruppo più piccolo di frasi che basta a fargli dire "supportato"	evidenza su testi senza annotazioni	`evidence/data/label_ablation.p`
Evidenza spostata	si allontanano le frasi gold, o si inseriscono frasi di disturbo	più casi di evidenza lontana	`evidence/data/augment_sparse.p`

Tutte scrivono righe nel formato del passo 2, con `gold_char_sets`, validate come le altre. Si generano **solo da righe di training**.

1. Documenti generati con posizioni note

Si parte da claim scomposti in fatti semplici (ricetta C2D di MiniCheck: ogni claim è riscritto come 2–4 frasi atomiche che insieme lo implicano). Al generatore (un LLM aperto in locale) si chiede:

```
Write a coherent news-style article of 15 to 40 sentences that includes each of the
following sentences word for word, each wrapped in its tag: <e1>…</e1>, <e2>…</e2>, …
You may place a tagged sentence inside a longer sentence of yours.
Add other context sentences without tags. Do not restate the information of the
tagged sentences anywhere else in the article. Place the tagged sentences far apart.

<e1>{fatto 1}</e1>
<e2>{fatto 2}</e2>
```

Poi:

1. ogni `<ei>…</ei>` deve comparire una volta sola e parola per parola, altrimenti la riga si scarta;
2. si tolgono i tag, registrando l'intervallo di caratteri di ogni testo marcato: quello è l'intervallo gold, **esatto**, anche se il generatore l'ha messo dentro una frase più lunga;
3. un giudice LLM diverso conferma: "do these passages, taken alone, establish the claim? yes/no"; si tengono solo i "yes".

Varianti dallo stesso articolo:

- **negativo**: si toglie un testo marcato necessario (e la sua frase) → il claim non è più supportato, `evidence_term = mil_neg`;
- **claim parziale**: il claim si riscrive usando solo alcuni fatti → supportato, con gold = solo quei testi marcati.

Queste sono le uniche righe in cui l'etichetta dice esattamente quale parte di una frase conta: sono il segnale migliore per insegnare alla testa tratti più stretti di una frase (esperimento H10, 11). `evidence_quality = construct`, peso 1.

2. Etichette ricavate col modello (ablazione)

Per righe supportate senza evidenza annotata (per esempio i dati C2D e D2C di MiniCheck come rilasciati). Si usa il verdetto di KnowLine-4B con il prompt con preambolo. Togliere pezzi di documento richiede di dividerlo in unità: si usano le frasi del nostro divisore. È un lavoro offline sui dati di training, che non entra in produzione.

```python
def ablation_label(sentences, p_supported, thr, max_k=6):
    """sentences: intervalli di caratteri delle frasi del documento;
    p_supported(indici) = P(supportato) con il documento ridotto a quelle frasi (nell'ordine originale).
    -> (indici delle frasi di evidenza, qualità) oppure None."""
    n, chosen = len(sentences), []
    while len(chosen) < max_k:
        best = max((k for k in range(n) if k not in chosen), key=lambda k: p_supported(sorted(chosen + [k])))
        chosen.append(best)
        if p_supported(sorted(chosen)) > thr:
            break
    else:
        return None                                   # nessun gruppo di al massimo 6 frasi basta: riga scartata
    for k in list(chosen):                                # pulizia: via le frasi che non servono
        rest = [j for j in chosen if j != k]
        if rest and p_supported(sorted(rest)) > thr:
            chosen = rest
    complement = [k for k in range(n) if k not in chosen]
    redundant = p_supported(complement) > thr             # l'evidenza è ripetuta altrove nel documento
    return sorted(chosen), ("ablation_partial" if redundant else "ablation")
```

- Ogni passo greedy manda tutti i documenti ridotti in un batch; KnowLine è causale, quindi si riusa il calcolo della parte iniziale comune del prompt.
- `thr` = soglia del verdetto.
- Filtri: un giudice LLM conferma le frasi trovate (atteso almeno l'80% di conferme); su 300 righe si rifà la ricerca con elephant-1, e i gruppi devono coincidere in buona parte (Jaccard medio ≥ 0,7).
- Le frasi trovate diventano `gold_char_sets`; peso della riga 0,5 (`ablation_partial`: 0,3).

3. Evidenza spostata

Da righe con almeno 2 frasi gold:

Trasformazione	Come
Agli estremi	la prima frase gold va all'inizio del documento, l'ultima alla fine
Frasi di disturbo	5–40 frasi di altri documenti di training della stessa fonte inserite fra le frasi gold
Lontano dall'inizio	testo di disturbo aggiunto in testa finché la prima frase gold è oltre 1.024 token (o oltre 4.096)
Negativi spostati	le stesse operazioni su righe non supportate

Si spostano frasi intere della fonte (gli intervalli gold), non token: il documento nuovo si ricostruisce pezzo per pezzo con `DocBuilder` (`evidence/data/schema.py`), che restituisce i nuovi intervalli di caratteri.

Controlli:

- una frase gold che inizia riferendosi alla precedente ("He", "It", "This", "The company"…) non si sposta da sola: si sposta con la precedente, o la riga si scarta;
- il giudice conferma che l'etichetta non è cambiata;
- le frasi di disturbo vengono solo da documenti di training.

4. Quante righe

Famiglia	Prima prova	Versione completa
Generati con posizioni esatte	2.000	10.000
Etichettati col modello	2.000	10.000
Evidenza spostata (positivi)	4.000	10.000
Negativi spostati	1.000	3.000

5. Controlli

Controllo	Atteso
30 righe per famiglia lette a mano: le parti gold dimostrano il claim e non ne manca nessuna necessaria	almeno 27 su 30
Conferme del giudice sull'ablazione	≥ 80%
Accordo KnowLine–elephant-1 sull'ablazione	Jaccard ≥ 0,7
Righe spostate con riferimenti rotti (30 lette)	0

Passo 4 — Split e mix di training

In breve

1. **Split**: si decide una volta quali righe servono per addestrare (train), per scegliere (dev) e per la misura finale (test), e non si cambia più. Si divide per documento (`doc_id`): lo stesso documento non può stare in due split.
2. **Mix** `ev_tok_head`: le righe di training con il loro tipo di loss e i loro pesi.

Script: `evidence/data/make_splits.py`, `evidence/data/make_mix.py`. Uscita in `dataset/evidence/splits/` e `dataset/evidence/mixes/`, con un `manifest.json` (hash dei file, conteggi, seme 13).

1. Regole degli split

Fonte	Train	Dev	Test
SciFact	90% del train d'origine (per `doc_id`)	10% del train d'origine	validation d'origine
HoVer	95% del train d'origine (per `doc_id`)	5% del train d'origine, solo righe `exact`	dev d'origine, solo righe `exact`
WiCE	train d'origine **senza** le righe con `aggrefact_overlap`	—	dev + test d'origine (tutte con `aggrefact_overlap`): test fuori dominio

Perché:

- **WiCE**: le pagine del suo dev e test sono le stesse del dev e test di LLM-AggreFact, il benchmark finale; addestrare su di esse falserebbe quella misura. Restano utili come test fuori dominio (pagine web, non Wikipedia).
- **HoVer** **`db_split`** (pagine prese dal db, etichetta sfasata nel 2% dei casi): si usano in training con peso minore, ma non per misurare.
- I `doc_id` di dev e test non compaiono mai nel train; i dati generati e spostati nascono solo dal train.

2. I set

Set	Contenuto	Uso
`train`	il resto	training della testa
`dev_evid_select`	metà dei positivi di dev (SciFact, HoVer), divisa per `doc_id`	tutte le scelte: configurazione, decodifica, esperimenti
`dev_evid_confirm`	l'altra metà	conferma delle scelte

Set	Contenuto	Uso
`dev_evid_neg`	negativi del dev con documenti lunghi	controllare che la testa non accenda tratti su claim non supportati
`test_evid_id`	test di HoVer (`exact`) e SciFact	misura finale
`test_evid_ood`	WiCE dev + test	misura finale fuori dominio
`test_evid_sparse`	da `test_evid_id`, con l'evidenza spostata lontano (passo 3, solo con frasi di disturbo del test)	evidenza lontana
`test_nopunct`	`test_evid_id` con la punteggiatura tolta dal documento (gli intervalli gold si ricalcolano)	documento senza frasi riconoscibili

Per ogni set si salvano: frasi gold per riga, distanza fra la prima e l'ultima frase gold (in frasi e in token), lunghezza in token.

3. Il mix `ev_tok_head`

Ogni riga riceve il tipo di loss dal suo `evidence_term` (08 §3) e due pesi:

- **peso della riga** (`row_weight`): quanto conta la riga nella media del batch;
- **ω** (`omega`): quanto pesano i token **fuori** dalle frasi gold. Basso dove le annotazioni sono incomplete (frasi valide non segnate).

Famiglia	Termine	Peso della riga	ω
HoVer supportati, `exact`	`supervised`	1,0	0,5
HoVer supportati, `db_split`	`supervised`	0,5	0,5
SciFact SUPPORT	`supervised`	1,0	0,5
WiCE supportati (train)	`supervised`	1,0	0,3
Generati con posizioni esatte (passo 3)	`supervised`	1,0	1,0
Etichettati col modello (passo 3)	`supervised`	0,5	0,5
Negativi: HoVer NOT_SUPPORTED, SciFact CONTRADICT/NEI, WiCE not_supported	`mil_neg`	1,0	—
WiCE parzialmente supportati	`none`: fuori dal mix della testa	—	—

Con le sole tre fonti il mix ha circa 14.000 positivi e 10.000 negativi (HoVer ne è la gran parte).

Vincoli che lo script controlla e stampa:

Vincolo	Valore
Negativi	20–45% delle righe (con le tre fonti sono ~40%: si può sottocampionare a 30%)

Vincolo	Valore
Positivi con 2 o più frasi gold	≥ 40%
Positivi con evidenza oltre 1.024 token dall'inizio del documento	≥ 15%
Documenti in comune con dev o test	0
Righe WiCE con `aggrefact_overlap`	0

4. Comandi

```
python -m evidence.data.make_splits --out dataset/evidence/splits
python -m evidence.data.make_mix --recipe recipes/ev_tok_head.json --out
↪  dataset/evidence/mixes/ev_tok_head.jsonl
```

La ricetta (`recipes/ev_tok_head.json`) contiene famiglie, numeri, pesi e seme: lo stesso mix si ricostruisce identico.

Passo 5 — Il prompt con il preambolo

In breve

Si aggiunge una riga con il claim **prima** dello stato (documento e claim), lasciando il resto del prompt identico a quello del fine-tuning binario. Poi si controlla che il verdetto funzioni ancora con questa riga in più: se sì, si riusa così com'è; se no, si riaddestra con la stessa ricetta del fine-tuning binario, includendo la riga.

1. Perché serve

Nei modelli causali (KnowLine-4B, OneJev, Nimble) ogni token vede solo i token che lo precedono. Nel prompt del fine-tuning binario lo stato è "documento, poi claim": quando il modello legge il documento non sa ancora che cosa deve verificare, e i vettori dei token del documento (quelli che la testa usa) non dipendono dal claim. Con il claim scritto anche prima, ogni token del documento è letto "sapendo cosa si cerca".

Nei modelli bidirezionali (elephant-1, StartLux, d3-flash, pplx, Jeffan) i token del documento vedono il claim negli 8 layer ad attenzione piena anche senza preambolo, ma non nei 24 layer DeltaNet, che restano causali. Il preambolo serve meno; l'esperimento H8 (11) misura quanto.

Il claim resta anche nella posizione originale, dopo il documento: lì il verdetto è stato addestrato a vederlo.

2. Esempio (KnowLine-4B, schema)

Il testo esatto lo produce il front end degli autori (`knowline_layout.py`). Schema:

```
senza preambolo
[turno utente]       document: The meeting was moved to Tuesday at 10:00. It will be held in Rome.
                     claim: The meeting is on Tuesday.
[turno utente]       <istruzione> Question: <domanda> Options: A. Yes: … B. No: …
[turno assistente]   Answer:

con preambolo
[turno utente]       Claim under review: The meeting is on Tuesday.

                     document: The meeting was moved to Tuesday at 10:00. It will be held in Rome.
                     claim: The meeting is on Tuesday.
[turno utente]       <istruzione> Question: <domanda> Options: A. Yes: … B. No: …
[turno assistente]   Answer:
```

KnowLine scrive **due volte** le richieste fino a 3.000 token ("Read the same input again before answering:"). Il preambolo fa parte dello stato e compare in entrambe le copie. I token del

documento per la testa si prendono dalla **seconda** copia: ha già visto tutto il documento e il claim della prima, quindi è la lettura più informata.

3. Dove va il preambolo negli altri modelli

Regola unica: **nello stesso turno dello stato, subito prima dello stato**, con lo stesso escaping usato per lo stato (per gli stati in JSON, il claim scritto come stringa JSON).

Modello	Lo stato nel prompt	Dove va il preambolo
KnowLine-4B	testo indentato in un turno utente	prima dello stato
elephant-1	`<state>` JSON indentato `</state>` nel prefisso comune	prima di `<state>`: resta nel prefisso, visto da tutte le domande
StartLux 9B/4B	`Evidence:` + JSON compatto	prima di `Evidence:`
d3-flash, Jeffan-4B	`State:` + JSON	prima di `State:`
OneJev1.6-9B	blocco dello stato nel prefisso comune	prima del blocco
Nimble-9B-v3	turno utente JSON con contesto e schema	campo testuale prima del contesto, se il motore lo accetta; altrimenti Nimble esce dagli esperimenti sul preambolo
pplx-decider-flash	sequenza unica con stato e domande	prima dello stato
Clef-Flash	`STATE:` + JSON compatto	prima di `STATE:`

Si implementa come opzione `preamble=` del renderer di ciascun modello (`evidence/token_render.py`), che chiama il codice degli autori. Test: con `preamble=None` il prompt è identico token per token a quello del fine-tuning binario; con il preambolo, i token dopo il preambolo sono identici (al più un token di differenza al confine).

4. Scegliere il testo del preambolo

Due candidati:

- **P-A**: `Claim under review: <claim>`
- **P-B**: `The following document will be checked against this claim: <claim>`

Sul dev del fine-tuning binario (`dev_select`), per ciascun modello, si misurano BAcc, AUC ed ECE in sei condizioni:

Condizione	Modello del verdetto	Prompt
B-1D	di partenza (prima del fine-tuning binario)	senza preambolo
B-SAND-A, B-SAND-B	di partenza	P-A, P-B
B-FT	fine-tunato (fine-tuning binario)	senza preambolo
B-FT-SAND-A, B-FT-SAND-B	fine-tunato	P-A, P-B

Si sceglie il testo con cui B-FT-SAND è più vicino a B-FT in AUC; a parità (entro 0,2 punti) il più corto. Lo stesso testo per tutti i modelli.

5. Riusare il verdetto o riaddestrarlo

Risultato su `dev_select`	Cosa fare
B-FT-SAND entro 0,3 punti di BAcc e AUC da B-FT, ECE entro 0,01	il verdetto fine-tunato si usa così com'è; si ricalcola solo la soglia, se si sposta
peggio di così	si riaddestra la LoRA del verdetto con la ricetta del fine-tuning binario e il preambolo nel prompt

6. Dove si scrive la scelta

In `question.json` (il file della domanda congelata del fine-tuning binario) si aggiungono due campi, letti da tutti gli script:

```
"preamble": {"template": "Claim under review: {claim}", "chosen_on": "dev_select"},
"verdict":  {"knowline-4b": "binary_plan", "elephant-1": "retrained_with_preamble"}
```

(Valori di esempio.)

Passo 6 — Dai caratteri ai token, e salvataggio dei vettori

In breve

1. **Rendering** (`evidence/token_render.py`): per ogni riga si costruisce il prompt con il preambolo (codice degli autori) e si calcola, per **ogni token del documento**, a quali caratteri del documento originale corrisponde (`tok_char`). Da qui l'etichetta di ogni token (dentro o fuori da una frase gold) e, in produzione, la conversione dei tratti in caratteri.
2. **Estrazione** (`evidence/token_extract.py`): un forward per riga con gli hook del passo 01; si salvano su disco i vettori ridotti di ogni token, la sua attenzione e `q`. Il modello è congelato: si calcolano una volta sola.

1. L'esempio renderizzato

```
{"id": "hover:train:…", "model": "knowline-4b",
 "input_ids": [248045, 846, "…"], "answer_pos": 2371,
 "doc_tok": [1103, 2290],
 "tok_char": [[0, 7], [7, 11], [11, 19], "…"],
 "tok_label": [0, 0, 1, 1, 1, "…"],
 "gold_tok_spans": [[37, 61], [612, 640]],
 "label": 1, "evidence_term": "supervised", "row_weight": 1.0, "omega": 0.5}
```

Campo	Significato
`doc_tok`	primo e ultimo token del documento nel prompt (KnowLine letto due volte: della seconda copia)
`tok_char[i]`	caratteri `[inizio, fine)` del documento **originale** coperti dal token `doc_tok[0] + i`
`tok_label[i]`	1 se il token sta dentro una frase gold (§3)
`gold_tok_spans`	per ogni frase gold dell'insieme usato in training, il primo e l'ultimo token con etichetta 1 (servono a `L_hit`, 08 §3)

Con più insiemi alternativi (WiCE), in training si usa l'**unione** delle frasi gold: tutte sono evidenza valida.

2. Dai token ai caratteri del documento originale

Nel prompt il documento può essere scritto diversamente dall'originale. Per i modelli con lo stato in JSON (elephant-1, StartLux, d3-flash, Jeffan, Nimble, pplx, Clef) virgolette e a capo diventano sequenze di due caratteri (" → \", a capo → \n). Per KnowLine lo stato è testo indentato. Serve

quindi, per ogni carattere del documento **come appare nel prompt**, il carattere originale da cui viene.

Procedura:

1. costruire il prompt in tre pezzi: prima del documento, documento come appare nel prompt, dopo il documento (si conosce così `doc_start`, dove inizia il documento nel prompt);
2. per il pezzo del documento, `escape_with_map` restituisce il testo trasformato e, per ogni suo carattere, l'indice del carattere originale (`origin`);
3. tokenizzare il prompt con `return_offsets_mapping=True`;
4. `token_char_spans`: per ogni token che cade nel pezzo del documento, `tok_char` = dal primo all'ultimo carattere originale che copre.

```
ESC = {'"': '\\"', "\\": "\\\\", "\n": "\\n", "\r": "\\r", "\t": "\\t", "\b": "\\b", "\f": "\\f"}   # come
↪ json.dumps


def escape_with_map(text, escape_lt=False):
    """Il testo come appare in una stringa JSON e, per ogni carattere del testo trasformato, il carattere
    ↪ originale da cui viene."""
    out, origin = [], []
    for i, ch in enumerate(text):
        if ch in ESC:
            piece = ESC[ch]
        elif escape_lt and ch == "<":
            piece = "\\u003c"
        elif ord(ch) < 0x20:
            piece = f"\\u{ord(ch):04x}"
        else:
            piece = ch
        out.append(piece)
        origin.extend([i] * len(piece))
    return "".join(out), origin


def token_char_spans(offsets, doc_start, doc_text_in_prompt_len, origin):
    """offsets: (inizio, fine) di ogni token nel prompt (return_offsets_mapping); doc_start: dove inizia il
    ↪ documento nel prompt;
    origin: per ogni carattere del documento nel prompt, il carattere originale. -> (indici dei token del
↪ documento, tok_char)."""
    doc_end = doc_start + doc_text_in_prompt_len
    idx, spans = [], []
    for k, (a, b) in enumerate(offsets):
        a, b = max(a, doc_start), min(b, doc_end)
        if a >= b:
            continue
        idx.append(k)
        spans.append((origin[a - doc_start], origin[b - 1 - doc_start] + 1))
    return idx, spans


def token_labels(tok_char, gold_char_spans, text):
    """1 se almeno metà dei caratteri non spazio del token sta dentro un intervallo gold."""
    labels = []
    for a, b in tok_char:
        chars = [c for c in range(a, b) if not text[c].isspace()]
        inside = sum(any(g0 <= c < g1 for g0, g1 in gold_char_spans) for c in chars)
        labels.append(int(bool(chars) and inside * 2 >= len(chars)))
    return labels
```

Il codice è stato eseguito su un documento con virgolette e a capo dentro un JSON: il testo trasformato è identico a `json.dumps`, `\"` torna a `"`, `\n` torna all'a capo, e le etichette dei token seguono la frase gold. Esempio: documento `Rose "Byrne" is an\nAustralian actress. She starred in films.`, frase gold la prima; i token `Rose " Byrne " is an Australian actress .` hanno etichetta 1, `\n` e `She starred in films .` 0.

Casi da gestire:

- **KnowLine** non usa JSON: il documento compare quasi così com'è (con l'indentazione delle righe). `escape_with_map` si sostituisce con la trasformazione del suo `knowline_layout`, registrando allo

stesso modo `origin`. Nelle richieste lette due volte si prende la seconda copia del documento (`doc_start` della seconda).
- **Prefisso e domanda tokenizzati separatamente** (elephant-1, OneJev, Nimble): gli offset si calcolano sui pezzi e si sommano.
- **Token a cavallo** fra il documento e il testo intorno (rari, ai bordi): vengono tagliati all'intervallo del documento.

`escape_with_map` sostituisce quella usata dai dati del passo 2 per lo stesso scopo: un solo file, `evidence/escape.py`.

3. L'etichetta di un token

```
tok_label[i] = 1  se almeno metà dei caratteri non spazio del token sta dentro un intervallo di gold_char_sets
             = 0  altrimenti (e sempre 0 se la riga ha label 0)
```

Le fonti annotano frasi intere: in una frase gold anche le parole irrilevanti hanno etichetta 1. La loss tiene conto di questo rumore (08 §2).

4. Che cosa si salva (fase 1a)

Feature	Forma	Tipo	Byte per token
`h`	n × 2 × 256	fp16, proiettato con la PCA (layer 23 e 31)	1.024
`att`	n × 128	fp16	256
`tok_char`	n × 2	int32	8
`tok_label`	n	int8	1
`q`	2 × D	fp32	— (uno per riga)
`answer_logits`	2	fp32	—

Per una riga HoVer (~1.500 token) circa 2 MB; per le ~28.600 righe delle tre fonti circa 60 GB.

Formato: `data/evidence_tok/features/<modello>/<set>/shard-00000.safetensors`, 500 righe per file (tensori concatenati lungo i token) più `index.jsonl` (`id`, shard, primo token, n, `gold_tok_spans`, `evidence_term`, pesi). Ogni cartella ha un `meta.json` con modello, revisione dei pesi, preambolo, layer, file della PCA, hash di `token_render.py` ed `escape.py`: il training rifiuta vettori calcolati con impostazioni diverse.

5. Comando

```
gpu-run --batch --name evtok-extract-knowline -- bash -c \
  'python -m evidence.token_extract --model knowline-4b \
     --rows dataset/evidence/mixes/ev_tok_head.jsonl dataset/evidence/splits/dev_evid_select.jsonl \
            dataset/evidence/splits/dev_evid_confirm.jsonl dataset/evidence/splits/dev_evid_neg.jsonl \
     --out data/evidence_tok/features/knowline-4b > logs/evtok-extract-knowline.out 2>&1'
```

Riprende dagli shard completi; ordina le righe per lunghezza (meno riempimento); batch 1 sopra i 4.096 token.

6. Test

Test	Deve dare
Testo ricostruito dai `tok_char` = documento originale (spazi a parte)	200 righe su 200, per ogni modello
Documenti con `"`, `\`, a capo, tab, `<`, emoji, testo non latino	`tok_char` giusti
Per ogni frase gold, i token con etichetta 1 la coprono (bordi a parte)	tutte le righe

Test	Deve dare
KnowLine letto due volte: `doc_tok` cade nella seconda copia	sempre
Senza preambolo, `input_ids` identici al fine-tuning binario	sempre
P(supportato) dai logit salvati = verdetto scelto al passo 5	differenza < 0,005

Passo 7 — Metodi semplici da battere

In breve

Prima di addestrare la testa si misura quanto ottengono metodi semplici, sugli stessi set, con le stesse metriche (10) e la stessa decodifica in tratti (08 §5). Servono a due cose: sapere se la testa vale la pena (deve batterli con margine) e capire quanto è difficile il problema sui nostri dati, soprattutto con l'evidenza lontana.

Script: `evidence/token_baselines.py`. Ogni metodo dà un punteggio per token; per i metodi senza soglia propria, la soglia si sceglie su metà del dev (A) e si misura sull'altra metà (B).

1. I metodi

Nome	Punteggio del token t	Costo in produzione
E-RAND	casuale	nullo; è il pavimento
E-LEX	1 se la parola del token (minuscola, senza stopword) compare nel claim, altrimenti 0; poi media mobile su 15 token	nullo; è il riferimento principale
E-EMB	somiglianza (coseno) fra l'embedding del claim e quello della finestra di 30 token centrata su t, con un modello di embedding piccolo (`BAAI/bge-small-en-v1.5`)	un encoder in più
E-ATT	attenzione della posizione di risposta verso t (feature `att` del passo 6); combinazione delle 8 coppie (layer, testa) migliori su A; media mobile su 9 token	nullo: legge feature già salvate
E-ABL	frasi trovate con l'ablazione del passo 3 (molti forward), tutti i loro token a 1	centinaia di forward per riga: solo riferimento

Che cosa dicono:

- **E-LEX** ed **E-EMB** misurano quanto si ottiene con la sola somiglianza al claim. Non possono trovare le parti "ponte", che non condividono parole con il claim (nell'esempio di Nancy Howell, la frase sul seminario): sull'evidenza lontana ci si aspetta che crollino. Se la testa non li batte con margine, l'utente avrebbe quasi lo stesso con un confronto di parole.
- **E-ATT** è per natura una baseline per token: dice se il modello già "guarda" l'evidenza, e quanto aggiunge addestrare una testa.
- **E-ABL** è il riferimento "di qualità": fedele al modello per costruzione, ma costa centinaia di forward. Dice quanto ci si avvicina con uno solo.

2. Come si sceglie la combinazione di E-ATT

1. Si divide `dev_evid_select` in A e B per `doc_id`.
2. Su A, per ognuna delle 128 coppie (layer pieno, testa): AUPRC per token dell'attenzione rispetto alle etichette.
3. Si tengono le 8 coppie migliori; il punteggio è la media dei loro ranghi normalizzati, poi media mobile su 9 token.
4. Si misura su B.

3. Comandi

```
python -m evidence.token_baselines --method rand lex emb --block dev_evid_select
python -m evidence.token_baselines --method att --model knowline-4b --block dev_evid_select
gpu-run --batch --name evtok-eabl -- bash -c \
  'python -m evidence.token_baselines --method abl --model knowline-4b --block dev_evid_select --per-source 300 \
     > logs/evtok-eabl.out 2>&1'
```

E-ABL: 300 righe per fonte.

4. Tabella da compilare (una per modello)

Metodo	Frasi gold colpite HoVer	Insieme completo HoVer	F1 caratteri HoVer	Frasi colpite SciFact	Colpite con evidenza a ≥ 10 frasi	Colpite senza punteggiatura	Tratti per riga
E-RAND							
E-LEX							
E-EMB							
E-ATT							
E-ABL							

Passo 8 — L'architettura della testa, la loss, la decodifica

In breve

Questo passo descrive per intero la rete da implementare e addestrare, la *testa di evidenza per token*, e come si trasformano i suoi punteggi in tratti di testo. Tre file:

- `evidence/token_head.py`: la testa;
- `evidence/token_losses.py`: la loss;
- `evidence/decode.py`: dai punteggi ai tratti (la stessa funzione in valutazione e in produzione).

Tutto il codice di questo file è stato eseguito su dati finti: forme dei tensori, gradienti, righe di lunghezza diversa nello stesso batch, valori numerici, decodifica e un ciclo di training completo tornano. I numeri di parametri e tempi sono misurati, non stimati.

1. La rete in una figura

Notazione: B = righe nel batch, N = token del documento (riempiti fino al più lungo del batch), L = layer letti (2), D = dimensione del modello (2.560 per KnowLine-4B, 4.096 per elephant-1), d = 256, K = blocchi = ⌈N / 16⌉.

```
 INGRESSI (dal forward del decision model)
   h    [B, N, L, 256]   vettori dei token del documento, layer 23 e 31, già ridotti con la PCA
   att  [B, N, 128]      attenzione della posizione di risposta sul token (8 layer pieni × 16 teste)
   q    [B, L, D]        vettore della posizione di risposta, layer 23 e 31
   mask [B, N]           1 = token vero, 0 = riempimento
        │
```

```
A. MISCELA DEI LAYER    h̄[t] = LN( Σ_l softmax(α)_l · h[t, l] )          [B, N, 256]
                        q̄    = LN( Σ_l softmax(α)_l · q[l] )            [B, D]
        |
B. VETTORE DEL TOKEN    x[t] = W_h h̄[t] + W_a att[t] + W_q q̄           [B, N, 256]
        |
C. CONTESTO LOCALE      3 blocchi residui:  x ← x + GELU( Conv1d_k5,dil(LN(x)) ),  dil = 1, 2, 4
                        ogni token vede 29 token intorno                   [B, N, 256]
        |
D. BLOCCHI              b[k] = media dei token k·16 … k·16+15 (solo quelli veri)   [B, K, 256]
                        + posizione del blocco P[k] + tipo "blocco"
        |
E. CONTESTO GLOBALE     sequenza [x0, b[1], …, b[K]],  x0 = W_q q̄ + tipo "risposta"
                        transformer NON causale, 2 layer, 4 teste, FFN 1.024, pre-norm
                        → X0 [B, 256],  X_b [B, K, 256]: ogni blocco ha visto tutto il documento e q
        |
F. RITORNO AI TOKEN     z[t] = GELU( W_r [ x[t] ; X_b[blocco di t] ] )      [B, N, 256]
        |
G. PUNTEGGI             s[t] = w_o · z[t] + b_o        (−10.000 sui token di riempimento)    [B, N]
                        s0   = w_o · X0  + b_o         soglia del documento                   [B]
```

2. Gli ingressi

Tensore	KnowLine-4B	elephant-1	Da dove
`h`	[B, N, 2, 256]	[B, N, 2, 256]	hook sui layer 23 e 31, proiezione PCA fissa (01 §5–6)
`att`	[B, N, 128]	[B, N, 128]	riga di attenzione della posizione di risposta, 8 layer × 16 teste (01 §7)
`q`	[B, 2, 2.560]	[B, 2, 4.096]	stato della posizione di risposta, layer 23 e 31
`mask`	[B, N]	[B, N]	1 sui token del documento

N tipico: ~1.500 (HoVer), ~600 (SciFact), da poche centinaia a ~18.000 (WiCE).

3. Strato per strato

A. Miscela dei layer

- Due pesi appresi α (partono da 0, quindi 50/50); `softmax(α)` li rende positivi a somma 1. La stessa miscela vale per `h` e per `q`.
- Una LayerNorm per ciascuno: i layer hanno scale molto diverse (l'ultimo ha valori più grandi).
- Perché: l'ultimo layer è specializzato a predire la risposta; il layer 23 rappresenta meglio il contenuto. Dopo il training, i pesi α dicono quale serve di più (se uno va a ~1, si può tenere solo quello).

B. Vettore del token

- `W_h` (256 → 256): il contenuto del token.
- `W_a` (128 → 256): dove il modello ha guardato per decidere. Durante il training le 128 feature di un token si azzerano con probabilità 0,1 (dropout per token), così la testa non dipende da una sola testa di attenzione.
- `W_q` (D → 256): il claim e il verdetto, uguali per tutti i token della riga. Sommandolo, ogni token è "letto alla luce" del claim anche nei layer della testa.
- I token di riempimento si azzerano.

C. Contesto locale

- 3 convoluzioni 1D lungo i token, finestra 5, dilatazione 1, 2, 4: campo visivo di 1 + 4·(1 + 2 + 4) = 29 token.
- Ognuna dentro un blocco residuo pre-norm: `x ← x + GELU(Conv(LN(x)))`; i riempimenti si riazzerano dopo ogni blocco.
- Perché: la stessa parola è evidenza in una frase e non in un'altra; serve vedere le parole vicine. Le convoluzioni sono parallele (veloci su GPU) e non dipendono da confini di frase.

D. Blocchi

- I token si raggruppano in blocchi **fissi** di 16, per posizione (token 0–15, 16–31, …). Il vettore del blocco è la media dei suoi token veri. Un documento di 1.500 token dà 94 blocchi; uno di 18.000, 1.125.
- Ogni blocco riceve un vettore di posizione appreso `P[k]` (fino a 4.096 blocchi, cioè 65.536 token) e un vettore di "tipo".
- Perché: per far comunicare parti lontane serve un'attenzione su tutto il documento. Su migliaia di token costa troppo; su ~100–1.000 blocchi è trascurabile. I blocchi non tagliano il risultato: l'uscita resta per token.

E. Contesto globale

- Sequenza `[x0, b[1], …, b[K]]`: `x0` rappresenta la posizione di risposta (W_q q̄ + tipo "risposta").
- Transformer encoder **non causale**: ogni elemento vede tutti gli altri, in avanti e all'indietro. 2 layer, d = 256, 4 teste, feed-forward 1.024, pre-norm, dropout 0,1. Maschera di riempimento sui blocchi vuoti.
- Uscita: `X0` (la posizione di risposta, dopo aver visto tutti i blocchi) e `X_b` (ogni blocco, dopo aver visto tutti gli altri e `q`).
- Perché: è qui che "frase 1" e "frase 40" si vedono. `x0` porta il punto di vista del verdetto: sa quali parti hanno pesato.

F–G. Ritorno ai token e punteggi

- Ogni token concatena il suo vettore locale `x[t]` con il vettore globale del suo blocco `X_b[⌊t/16⌋]` → `W_r` (512 → 256) → GELU.
- Un'unica uscita lineare `w_o` (256 → 1) dà `s[t]` per ogni token e, applicata a `X0`, la soglia `s0`. Stessa uscita per entrambi: punteggi e soglia sono sulla stessa scala, e `s[t] − s0` ha senso.
- Un token è evidenza se `s[t] > s0`.

4. Parametri e costo (misurati)

Componente	Parametri KnowLine-4B	Parametri elephant-1
A. miscela e normalizzazioni	5.634	8.706
B. `W_h`	65.792	65.792
B. `W_a`	33.024	33.024
B. `W_q`	655.616	1.048.832
C. convoluzioni (+ norme)	985.344	985.344
D. posizione dei blocchi e tipi	1.049.088	1.049.088
E. transformer	1.579.520	1.579.520
F–G. ritorno e uscita	131.585	131.585
Totale	**4.505.603** (18 MB in fp32)	**4.901.891** (19,6 MB)

Costo per una riga di 2.000 token: convoluzioni ~4 GFLOP, transformer sui 125 blocchi ~0,2 GFLOP; la proiezione PCA negli hook ~5 GFLOP. Il forward del modello su quella riga (4B, letta due volte) costa ~30 TFLOP: la testa aggiunge meno dello 0,05% dei calcoli.

Tempo misurato della testa (GPU locale RTX 5090 Laptop, fp32, una riga da 6.000 token): **3,2 ms**.

5. Il codice della testa

```
import torch
import torch.nn as nn
import torch.nn.functional as F


class TokenEvidenceHead(nn.Module):
    def __init__(self, d_q, n_layers=2, d_in=256, n_att=128, d=256, block=16, kernel=5, conv_layers=3,
                 enc_layers=2, heads=4, max_blocks=4096, use_att=True, use_local=True, use_global=True,
↪   att_dropout=0.1):
        super().__init__()
        self.block, self.use_att, self.use_local, self.use_global = block, use_att, use_local, use_global
        self.att_dropout = att_dropout                                    # in training: feature di attenzione di
        ↪  un token azzerate
        self.mix = nn.Parameter(torch.zeros(n_layers))                    # pesi dei layer (softmax)
        self.ln_h, self.ln_q = nn.LayerNorm(d_in), nn.LayerNorm(d_q)
        self.proj_h, self.proj_a, self.proj_q = nn.Linear(d_in, d), nn.Linear(n_att, d), nn.Linear(d_q, d)
        self.convs = nn.ModuleList([nn.Conv1d(d, d, kernel, padding=2 ** i * (kernel // 2), dilation=2 ** i)
                                    for i in range(conv_layers)])         # contesto locale: ~30 token
        self.conv_norms = nn.ModuleList([nn.LayerNorm(d) for _ in range(conv_layers)])
        self.pos, self.kind = nn.Embedding(max_blocks, d), nn.Embedding(2, d)
        enc = nn.TransformerEncoderLayer(d, heads, 4 * d, dropout=0.1, batch_first=True, norm_first=True)
        self.encoder = nn.TransformerEncoder(enc, enc_layers)             # contesto globale, sui blocchi
        self.back = nn.Linear(2 * d, d)
        self.out = nn.Linear(d, 1)

    def mixed(self, x, ln):                                               # x: [..., L, D]
        return ln((self.mix.softmax(0)[:, None] * x).sum(-2))

    def forward(self, h, att, q, mask):
        """h: [B, N, L, 256]; att: [B, N, 128]; q: [B, L, Dq]; mask: [B, N] (1 = token del documento)
        -> s: [B, N] punteggi dei token, s0: [B] soglia del documento."""
        m = mask.unsqueeze(-1).float()
        qv = self.proj_q(self.mixed(q, self.ln_q))                        # [B, d]
        x = self.proj_h(self.mixed(h, self.ln_h)) + qv[:, None]
        if self.use_att:
            att = att.float()
            if self.training and self.att_dropout:                            # intere, per token: non dipendere da
            ↪  una sola testa
                att = att * (torch.rand_like(att[..., :1]) >= self.att_dropout)
            x = x + self.proj_a(att)
        x = x * m
        if self.use_local:
            for conv, norm in zip(self.convs, self.conv_norms):           # convoluzioni residue, padding a zero
                x = x + F.gelu(conv(norm(x).transpose(1, 2)).transpose(1, 2)) * m
        B, N, d = x.shape
        nb = -(-N // self.block)
        xb = F.pad(x, (0, 0, 0, nb * self.block - N)).view(B, nb, self.block, d)
        mb = F.pad(m, (0, 0, 0, nb * self.block - N)).view(B, nb, self.block, 1)
        blocks = (xb * mb).sum(2) / mb.sum(2).clamp_min(1)                # media dei token di ogni blocco
        block_mask = mb.sum(2).squeeze(-1) > 0                             # [B, nb]
        x0 = qv + self.kind.weight[0]
        if self.use_global:
            seq = torch.cat([x0[:, None], blocks + self.pos(torch.arange(nb, device=x.device)) +
↪   self.kind.weight[1]], 1)
            pad = torch.cat([torch.zeros_like(block_mask[:, :1]), ~block_mask], 1)
            X = self.encoder(seq, src_key_padding_mask=pad)
            X0, Xb = X[:, 0], X[:, 1:]
        else:
            X0, Xb = x0, blocks
        ctx = Xb.repeat_interleave(self.block, dim=1)[:, :N]              # ogni token riceve il contesto del suo
↪   blocco
        z = F.gelu(self.back(torch.cat([x, ctx], -1)))
        s = self.out(z).squeeze(-1).masked_fill(mask == 0, -1e4)
        s0 = self.out(X0).squeeze(-1)
        return s, s0
```

Configurazione di partenza dell'esperimento **H0**: un solo layer (31), `use_att=False, use_local=False, use_global=False`, cioè un punteggio per token dal suo vettore e da `q`. Gli esperimenti (11) accendono un pezzo alla volta.

6. La loss

6.1 Il problema delle etichette

Le fonti annotano **frasi**, non parole: in una frase gold tutti i token hanno etichetta 1, anche "is", "the" o una data accessoria. Se si chiedesse alla testa di accendere ogni token gold, imparerebbe a evidenziare sempre frasi intere, e una testa che evidenzia solo la parte utile verrebbe penalizzata.

Per questo la loss ha quattro termini con ruoli diversi:

- **`L_tok`**, per token, con peso moderato: dà la direzione generale;
- **`L_hit`**: ogni frase gold dev'essere **colpita**, cioè almeno un suo token chiaramente sopra soglia. È ciò che conta per l'utente (e per l'insieme completo nei casi multi-hop);
- **`L_out`**: **fuori** dalle frasi gold nessun token deve superare la soglia. Pesato con ω, perché le annotazioni possono essere incomplete (una frase valida non segnata);
- **`L_tv`**: token vicini con punteggi simili, per tratti contigui invece che a macchia di leopardo.

6.2 Le formule

Con `d[t] = s[t] − s0` (positivo = sopra soglia), per una riga:

```
supervised:  L = L_tok + 1,0 · L_hit + 0,5 · ω · L_out + 0,1 · L_tv
  L_tok = Σ_t w_t · BCE(σ(d[t]), y[t]) / Σ_t w_t          w_t = α = 3 sui token gold, ω sugli altri
  L_hit = media sulle frasi gold g di softplus(1 − max_{t ∈ g} d[t])
  L_out = softplus(1 + max_{t ∉ gold} d[t])
  L_tv  = media_t |σ(d[t+1]) − σ(d[t])|
mil_neg:     L = 0,3 · softplus(1 + max_t d[t])              claim non supportato: nessun token sopra soglia
none:        L = 0                                           nessuna loss di evidenza
```

Il margine 1 vuol dire "chiaramente sopra / sotto", non appena sopra. Le righe hanno lunghezze diverse: la loss si calcola **riga per riga** e si fa la media pesata con il peso della riga (`row_weight`, 04 §3).

6.3 Il codice

```python
import torch
import torch.nn.functional as F


def token_evidence_loss(s, s0, y, mask, gold_spans, term, omega, alpha=3.0, margin=1.0,
                        b_hit=1.0, b_out=0.5, b_tv=0.1, mu_neg=0.3):
    """Una riga alla volta (i documenti hanno lunghezze diverse).
    s: [N] punteggi dei token; s0: [] soglia; y: [N] etichette 0/1; mask: [N]; gold_spans: [(a, b)] token,
    estremi inclusi;
    term: "supervised" | "mil_neg" | "none"; omega: peso dei token fuori dalle frasi gold."""
    d = (s - s0)[mask.bool()]
    y = y[mask.bool()].float()
    if term == "none":
        return s.sum() * 0.0
    if term == "mil_neg":                                          # nessun token sopra la soglia
        return mu_neg * F.softplus(d.max() + margin)
    w = torch.where(y > 0, torch.full_like(d, alpha), torch.full_like(d, omega))
    l_tok = (F.binary_cross_entropy_with_logits(d, y, reduction="none") * w).sum() / w.sum()
    l_hit = torch.stack([F.softplus(margin - d[a:b + 1].max()) for a, b in gold_spans]).mean()  # ogni frase gold
    colpita
    outside = d[y == 0]
    l_out = F.softplus(outside.max() + margin) if outside.numel() else d.sum() * 0.0      # niente acceso fuori
    p = torch.sigmoid(d)
    l_tv = (p[1:] - p[:-1]).abs().mean() if p.numel() > 1 else d.sum() * 0.0            # tratti contigui
    return l_tok + b_hit * l_hit + b_out * omega * l_out + b_tv * l_tv
```

6.4 Esempio numerico (anche test automatico)

10 token, frase gold sui token 2–5, `s0 = 0`, ω = 0,5, punteggi `−3; −3; 2; 0,5; −0,5; 2; −3; −3; −3; 1,5`. La frase gold è colpita (il token 2 vale 2), ma il token 9, fuori, è acceso (1,5) e il token 4, dentro, è sotto soglia. Loss: **1,4015**. La stessa riga come negativo (`mil_neg`): **0,9146**, perché il token più alto (2) sta sopra soglia.

7. La decodifica: dai punteggi ai tratti

1. token con `s[t] − s0 > δ` (δ = 0 di default, scelto sul dev, 09 §2);
2. tratti di token consecutivi; si uniscono tratti separati da al massimo 3 token;
3. si tolgono i tratti sotto 4 token (parole isolate);
4. si tengono i 5 tratti con punteggio medio più alto, in ordine di posizione;
5. se il verdetto è "supportato" e non resta nulla, si mostra il tratto intorno al token migliore;
6. ogni tratto diventa `[inizio, fine)` di caratteri con `tok_char` (06 §1), allargato alla parola intera; il punteggio del tratto è σ(media di `s − s0`).

```
import math


def decode(s, s0, tok_char, text, delta=0.0, max_gap=3, min_len=4, max_spans=5, force_one=True):
    """s: punteggi dei token del documento (lista); s0: soglia; tok_char: [(inizio, fine)] in caratteri del
    ↪ documento.
    -> lista di {"start", "end", "score"} sul testo originale, in ordine di posizione."""
    on = [x - s0 > delta for x in s]
    runs, i, n = [], 0, len(s)
    while i < n:                                            # tratti consecutivi sopra soglia
        if on[i]:
            j = i
            while j + 1 < n and on[j + 1]:
                j += 1
            runs.append([i, j]); i = j + 1
        else:
            i += 1
    merged = []
    for a, b in runs:                                       # si chiudono i buchi piccoli
        if merged and a - merged[-1][1] - 1 <= max_gap:
            merged[-1][1] = b
        else:
            merged.append([a, b])
    merged = [r for r in merged if r[1] - r[0] + 1 >= min_len]
    if not merged and force_one and n:                      # supportato: almeno un tratto, intorno al token
    ↪ migliore
        k = max(range(n), key=lambda t: s[t])
        merged = [[max(0, k - min_len), min(n - 1, k + min_len)]]
    score = lambda r: sum(s[r[0]:r[1] + 1]) / (r[1] - r[0] + 1) - s0
    merged = sorted(sorted(merged, key=score, reverse=True)[:max_spans])
    out = []
    for a, b in merged:
        start, end = tok_char[a][0], tok_char[b][1]
        while start > 0 and not text[start - 1].isspace():  # si allarga alla parola intera
            start -= 1
        while end < len(text) and text[end].isalnum():
            end += 1
        while start < end and text[start].isspace():
            start += 1
        out.append({"start": start, "end": end, "score": 1 / (1 + math.exp(-score([a, b])))})
    return out
```

Esempio (anche test): testo "Rose Byrne is an Australian actress. She starred in Get Him to the Greek in 2010.", token 0–6 sopra soglia tranne il 3 (un buco di un token), token 12 sopra soglia da solo → un solo tratto: "Rose Byrne is an Australian actress." Il buco è chiuso, il token isolato è tolto.

I parametri (δ, buco massimo, lunghezza minima, numero massimo di tratti) si scelgono sul dev e si scrivono in `decision.json` (12).

8. Test (`tests/test_token_head.py`)

- forme dei tensori con righe di lunghezza diversa nello stesso batch; i token di riempimento hanno punteggio −10.000; i punteggi di una riga non cambiano se nel batch c'è una riga più lunga (a meno di 1e-4);
- l'esempio numerico della loss (tolleranza 1e-3); `none` dà zero e nessun gradiente;
- l'esempio della decodifica; un documento senza token sopra soglia, con `force_one`, dà un tratto; i tratti non escono dal testo;
- con tutte le opzioni spente la testa è la configurazione H0.

Passo 9 — Training

In breve

Si addestra la testa in tre modi, dal più semplice al più costoso; si passa al successivo solo se il precedente non basta.

Fase	Il decision model	Ingresso della testa	Quando
1a	congelato, non in memoria	vettori ridotti con la PCA, salvati al passo 6	quasi tutti gli esperimenti: ogni prova dura minuti
1b	congelato, in memoria	vettori completi ricalcolati a ogni passo; la testa impara la sua riduzione al posto della PCA	conferma (H9): la PCA perde informazione?
2	addestrato anche lui, con una LoRA	come 1b, con gradiente nel modello	solo se dopo gli esperimenti gli obiettivi non sono raggiunti

Nelle fasi 1a e 1b il verdetto **non può cambiare**: il modello non viene toccato.

Ogni prova scrive in `data/evidence_tok/runs/<modello>/<id>/`: `config.json` (opzioni della testa, della loss, iperparametri, hash delle feature e del mix), `head.safetensors`, `log.jsonl`, `dev_metrics.json`.

1. Fase 1a: il ciclo di training

Ogni riga del mix porta: le feature salvate (`h`, `att`, `q`), le etichette dei token (`tok_label`), gli intervalli di token delle frasi gold (`gold_tok_spans`), il tipo di loss (`evidence_term`), ω e il peso della riga.

1. **Batch per lunghezza**: le righe si ordinano per numero di token e si raggruppano a 16, così il riempimento è minimo; l'ordine dei batch è casuale e cambia a ogni epoca.
2. **Riempimento**: ogni batch si porta alla riga più lunga; `mask` segna i token veri. La testa è invariante al riempimento (verificato: differenza ~1e-7).
3. **Loss per riga** (08 §6), moltiplicata per il peso della riga; media sul batch divisa per la somma dei pesi delle righe con loss.
4. **Ottimizzazione**: AdamW, clipping del gradiente a 1, learning rate con riscaldamento lineare (primo 5% dei passi) e discesa a coseno.
5. **A fine epoca**: valutazione su `dev_evid_select` con la decodifica e le metriche del passo 10; si tiene il checkpoint migliore; ci si ferma dopo 2 epoche senza miglioramento.

```
import math
import random

import torch
import torch.nn.functional as F

from head import TokenEvidenceHead
from losses import token_evidence_loss


def collate(rows, device):
    """rows: dict con h [n, L, 256], att [n, 128], q [L, D], tok_label [n], gold_tok_spans, evidence_term, omega,
     ↪  row_weight.
    -> tensori riempiti fino al documento più lungo del batch."""
    n = max(r["h"].shape[0] for r in rows)
    B, L = len(rows), rows[0]["h"].shape[1]
    h = torch.zeros(B, n, L, rows[0]["h"].shape[2])
    att = torch.zeros(B, n, rows[0]["att"].shape[1])
```

```
    y = torch.zeros(B, n)
    mask = torch.zeros(B, n)
    for i, r in enumerate(rows):
        k = r["h"].shape[0]
        h[i, :k], att[i, :k], y[i, :k], mask[i, :k] = r["h"].float(), r["att"].float(), r["tok_label"].float(), 1
    q = torch.stack([r["q"].float() for r in rows])
    return h.to(device), att.to(device), q.to(device), y.to(device), mask.to(device)


def batches(rows, size, seed):
    """Righe di lunghezza simile insieme (meno riempimento), ordine dei batch casuale."""
    order = sorted(range(len(rows)), key=lambda i: rows[i]["h"].shape[0])
    groups = [order[i:i + size] for i in range(0, len(order), size)]
    random.Random(seed).shuffle(groups)
    return groups


def batch_loss(model, rows, device):
    h, att, q, y, mask = collate(rows, device)
    s, s0 = model(h, att, q, mask)
    losses = [token_evidence_loss(s[i], s0[i], y[i], mask[i], r["gold_tok_spans"], r["evidence_term"],
    ↪  r["omega"])
              * r["row_weight"] for i, r in enumerate(rows)]
    weight = sum(r["row_weight"] for r in rows if r["evidence_term"] != "none")
    return torch.stack(losses).sum() / max(weight, 1e-6)


def train(rows, dev_rows, d_q, epochs=10, batch=16, lr=5e-4, seed=13, device="cuda", patience=2, evaluate=None,
    ↪  **head_kw):
    torch.manual_seed(seed)
    model = TokenEvidenceHead(d_q=d_q, n_layers=rows[0]["h"].shape[1], **head_kw).to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=0.01)
    steps = epochs * math.ceil(len(rows) / batch)
    sched = torch.optim.lr_scheduler.LambdaLR(
        opt, lambda k: min(1.0, k / max(1, 0.05 * steps)) * 0.5 * (1 + math.cos(math.pi * min(k, steps) / steps)))
    best, best_state, bad = -1.0, None, 0
    for epoch in range(epochs):
        model.train()
        for idx in batches(rows, batch, seed + epoch):
            loss = batch_loss(model, [rows[i] for i in idx], device)
            opt.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
            sched.step()
        score = evaluate(model, dev_rows) if evaluate else -loss.item()
        print(f"epoch {epoch}  loss {loss.item():.4f}  dev {score:.4f}")
        if score > best:
            best, best_state, bad = score, {k: v.detach().clone() for k, v in model.state_dict().items()}, 0
        else:
            bad += 1
            if bad >= patience:
                break
    model.load_state_dict(best_state)
    return model, best
```

Questo ciclo è stato eseguito su feature finte (120 righe di 200–900 token, segnale nascosto in alcune componenti dei token gold): converge in poche epoche e trova le frasi gold.

Nel codice vero le righe si leggono dagli shard del passo 6 (`evidence/token_dataset.py`, che restituisce gli stessi dizionari) e la funzione `evaluate` usa `decode` e le metriche del passo 10.

Iperparametro	Valore
Learning rate	5e-4, riscaldamento 5%, coseno
Weight decay	0,01
Batch	16 righe raggruppate per lunghezza
Epoche	fino a 10, arresto dopo 2 senza miglioramento
Dropout	0,1 nel transformer; 0,1 sulle feature di attenzione (intere, per token)

Iperparametro	Valore
Criterio del checkpoint	media di frasi gold colpite e F1 sui caratteri su `dev_evid_select`
Seme	13 (3 semi per la configurazione finale: 13, 14, 15)

```
python -m evidence.token_train --phase 1a --model knowline-4b --features data/evidence_tok/features/knowline-4b \
    --mix ev_tok_head --config configs/tok_h0.json --run h0
```

Memoria: per una riga di 6.000 token la testa usa poche decine di MB. La fase 1a gira su qualunque GPU, anche su CPU per righe brevi.

Prima della prima prova: la testa deve imparare a memoria 64 righe (frasi gold colpite ≈ 1, nulla acceso fuori). Se non ci riesce, c'è un errore in `tok_char`, nelle etichette o nel codice.

2. Scegliere i parametri della decodifica

Dopo il training, su `dev_evid_select`, solo sulle righe supportate che il verdetto giudica supportate, griglia su:

Parametro	Valori
δ (scarto dalla soglia)	da −1 a 1, passo 0,1
buco massimo	0, 2, 3, 5 token
lunghezza minima	2, 4, 6 token
tratti massimi	3, 5, 8

Criterio: media di frasi gold colpite e F1 sui caratteri. Conferma su `dev_evid_confirm`: non più di 1 punto sotto. I valori vanno in `decision.json`.

3. Che cosa guardare durante il training

Cosa	Su	Segnale di problema	Cosa fare
Frasi gold colpite, F1 sui caratteri	`dev_evid_select`	non superano E-LEX dopo 2 epoche	controllare `tok_char` ed etichette prima di toccare la testa
Tratti per riga, token per tratto	`dev_evid_select`	paragrafi interi, o tanti frammenti	pesi di `L_out` e `L_tv`; decodifica
Frasi colpite per distanza fra le frasi gold	righe con evidenza lontana	crollo oltre 10 frasi	contesto globale (H3); più righe spostate
Righe con tratti sopra soglia	`dev_evid_neg`	oltre il 30%	peso di `mil_neg` più alto
Pesi α dei layer	testa	uno vicino a 1	basta quel layer (metà spazio su disco)

4. Fase 1b: vettori completi, riduzione appresa

Il modello è in memoria e congelato. A ogni passo si rifà il forward (senza gradiente nel modello) e si prendono i vettori **completi** dei token del documento (D numeri). La testa li riduce con una matrice appresa, `nn.Linear(D, 256)` per layer, **inizializzata con la PCA** (media e proiezione), al posto della proiezione fissa. Tutto il resto è uguale.

Impostazione	Gruppo 4B	Gruppo 9B
GPU	24 GB	45 GB
Righe per passo	2–4 (accumulando fino a 16)	1 (accumulando fino a 16)
Lunghezza massima	8.192 token (si escludono le righe oltre: oggi 75 di WiCE)	8.192 token
Partenza	la testa migliore della fase 1a	idem
Learning rate	2e-4 (proiezione 5e-5)	idem
Epoche	2	1

5. Fase 2: training anche del modello

Solo se gli obiettivi non sono raggiunti dopo gli esperimenti della fase 1. Il modello del verdetto è il decision model con la LoRA del fine-tuning binario (rango 16 sui moduli di attenzione, DeltaNet e MLP di Qwen3.5). Si continua ad addestrare quella LoRA insieme alla testa.

Impostazione	Valore
Partenza	LoRA del verdetto + testa migliore della fase 1
Learning rate	LoRA 5e-6 – 1e-5 (basso: il verdetto è già addestrato); testa 2e-4
Loss	loss del verdetto (cross-entropy sui due punteggi sì/no, su tutte le righe) + 0,3 × loss di evidenza
Dati	i dati del fine-tuning binario più le righe con evidenza
Epoche	1
GPU	4B: 24 GB; 9B: 45 GB

Il verdetto ora può cambiare, quindi:

- prima del primo passo si controlla che sia identico a quello di partenza;
- a ogni checkpoint si misurano BAcc, AUC ed ECE sul dev del fine-tuning binario; un checkpoint si accetta solo se BAcc resta entro 0,3 punti ed ECE entro 0,01;
- se la calibrazione si sposta, si aggiunge un termine che tiene le probabilità vicine a quelle del verdetto di partenza: $\beta \cdot KL(p_{partenza} \| p)$, $\beta = 0{,}1$.

6. Test prima di ogni nuova configurazione

Test	Atteso
Imparare a memoria 64 righe	frasi gold colpite ≈ 1, F1 sui caratteri > 0,9
H0 riprodotto con le opzioni spente	stessi numeri (stesso seme)
Righe `none`	loss e gradiente zero
Batch con righe di lunghezze molto diverse	stessi punteggi di una riga alla volta (differenza < 1e-4)

Passo 10 — Valutazione

In breve

Per ogni modello si misurano tre cose:

1. il **verdetto** non è cambiato;
2. i **tratti mostrati** sono quelli giusti, anche quando l’evidenza è lontana o il testo non ha punteggiatura;
3. la **latenza** resta quella di un decision model.

Script: `evidence/token_eval.py` (qualità), `evidence/latency.py` (velocità).

Le metriche di evidenza si calcolano sulle righe supportate **che il modello giudica supportate**: solo lì si mostrano tratti. A parte si riporta quante righe supportate restano senza evidenza perché il verdetto dice “no”.

1. Metriche sulle frasi gold (le principali)

Metrica	Definizione
Frasi gold colpite	quota di frasi gold con almeno metà dei caratteri (spazi esclusi) dentro i tratti mostrati
Insieme completo	quota di righe in cui tutte le frasi gold (di almeno un insieme) sono colpite: per l’evidenza multi-hop è quella che conta, metà evidenza non convince
Colpite per distanza fra frasi gold	fasce: vicine / 2–9 / 10–19 / ≥ 20 frasi fra la prima e l’ultima
Colpite per distanza dall’inizio del documento	< 1.024 / 1.024–4.096 / > 4.096 token
Colpite senza punteggiatura	sul set `test_nopunct` (04 §2)
Sufficienza	il documento ridotto ai soli tratti mostrati (uniti da ” … “) fa ancora dire”supportato” al modello
Necessità	il documento senza i tratti mostrati fa dire “non supportato” (si riporta, non è un obiettivo: se l’evidenza è ripetuta può fallire anche quando è giusta)
Negativi accesi	su `dev_evid_neg`: righe con almeno un tratto sopra soglia
Tratti per riga, token per tratto	media e 95° percentile

Sufficienza e necessità costano 2 forward per riga: si calcolano solo in valutazione.

2. Metriche sui caratteri

Metrica	Definizione
Precisione	quota dei caratteri mostrati che sta dentro una frase gold
Recall	quota dei caratteri delle frasi gold che è mostrata
F1 sui caratteri	media armonica, per riga, poi media sulle righe
AUPRC per token	dai punteggi `s[t] − s0`, prima della decodifica: quanto bene ordinano i token

Con più insiemi alternativi (WiCE) si prende, per riga, l’insieme con l’F1 più alto.

La recall sui caratteri penalizza una testa che evidenzia solo la parte utile di una frase gold (le etichette sono frasi intere): per questo non è la metrica principale.

```python
def _chars(spans, text):
    return {c for a, b in spans for c in range(a, b) if not text[c].isspace()}


def evidence_metrics(pred_spans, gold_sets, text, hit_share=0.5):
    """pred_spans: [(inizio, fine)] mostrati; gold_sets: insiemi alternativi di [(inizio, fine)] frasi gold.
    -> metriche della riga, con l'insieme gold che dà l'F1 più alto."""
    pred = _chars(pred_spans, text)
    best = None
    for gold in gold_sets:
        g = _chars(gold, text)
```

```
        tp = len(pred & g)
        p = tp / len(pred) if pred else 0.0
        r = tp / len(g) if g else 0.0
        f1 = 2 * p * r / (p + r) if p + r else 0.0
        hits = [len(pred & _chars([s], text)) >= hit_share * len(_chars([s], text)) for s in gold]
        m = {"precision": p, "recall": r, "f1": f1, "gold_hit": sum(hits) / len(hits), "full_set": all(hits)}
        if best is None or m["f1"] > best["f1"]:
            best = m
    return best
```

Esempio (eseguito): documento `Rose "Byrne" is an\nAustralian actress. She starred in films.`, frase gold la prima. Tratto mostrato "Rose "Byrne" is an": precisione 1, recall 0,48, F1 0,65, frase **non** colpita (meno di metà dei caratteri). Tratto = tutto il documento, con due insiemi alternativi: si sceglie l'insieme migliore, frase colpita, F1 0,79.

3. Confronto con una testa per frase

L'alternativa a questa testa è una **testa per frase**: si divide il documento in frasi con un divisore (NLTK), si riassume ogni frase in un vettore e si dà un punteggio a ogni frase. È più semplice, ma lega il modello al divisore, anche in produzione.

Per confrontarle sullo stesso terreno (solo in valutazione) i tratti si convertono in frasi: una frase della riga (`sentences`, frasi del nostro divisore già salvate nei dati) è "mostrata" se almeno metà dei suoi caratteri è dentro un tratto. Poi si calcolano, per entrambe le teste:

- precisione, recall, F1 per frase;
- insieme completo (tutte le frasi gold mostrate);
- le stesse metriche su `test_nopunct`, dove il divisore non trova le frasi e la testa per frase si degrada.

Il divisore qui serve solo a misurare: non entra nel modello.

4. Prove di comportamento

Prova	Atteso
Evidenza spostata all'inizio / a metà / alla fine	stesse frasi gold colpite in ≥ 90% delle righe
Evidenza oltre 1.024 e oltre 4.096 token	idem
Frasi di disturbo aggiunte	nessun tratto dentro il disturbo in ≥ 95%
Documento senza punteggiatura	frasi gold colpite al massimo 5 punti sotto il documento normale
Frase gold riscritta senza parole in comune col claim	colpita in ≥ 80%
Coppia quasi identica (positivo / negativo con un fatto tolto)	positivo: tratti giusti; negativo: nessun tratto sopra soglia

5. Latenza

Stesso server (12), stessa GPU, 1.000 richieste reali di lunghezze diverse (le prime 50 non si contano):

	Prompt	Uscita
A	senza preambolo	solo verdetto
B	con preambolo	solo verdetto
C	con preambolo	verdetto + tratti

Obiettivo: C ≤ B + 3% (modelli 4B: +3% o +3 ms). La differenza B − A è il costo del preambolo e si riporta a parte. Si misurano anche, separatamente, la proiezione negli hook, la testa (3,2 ms misurati per 6.000 token su RTX 5090 Laptop) e la decodifica.

6. Comandi

```
python -m evidence.token_eval --model knowline-4b --run data/evidence_tok/runs/knowline-4b/<id> --block
↪  dev_evid_select
# test: una volta sola, a configurazione congelata
gpu-run --batch --name evtok-test -- bash -c \
  'python -m evidence.token_eval --model knowline-4b --release <rilascio> \
     --block test_evid_id test_evid_ood test_evid_sparse test_nopunct --sufficiency > logs/evtok-test.out 2>&1'
```

Lo script salva `metrics.json` e `predictions.jsonl` (per riga: tratti, soglia, punteggi dei token compressi), così le metriche si ricalcolano senza rifare i forward.

7. Report finale (uno per modello)

Set	Metrica	E-LEX	E-ATT	E-ABL	Testa per frase	Testa per token
test_evid_id	frasi colpite / insieme completo / F1 caratteri					
test_evid_ood	idem					
test_evid_sparse	frasi colpite a ≥ 20 frasi di distanza					
test_nopunct	frasi colpite					
tutti	sufficienza					
dev_evid_neg	negativi accesi					
verdetto	BAcc (uguale al fine-tuning binario?)	—	—	—		
latenza	p50 / p95, C contro B					

Passo 11 — Esperimenti

In breve

Si parte da una testa minima (H0) e si aggiunge **un pezzo alla volta**, tenendolo solo se migliora. Tutto si fa su **KnowLine-4B** con i vettori salvati (fase 1a): ogni prova è un training della sola testa, di pochi minuti. Alla fine elephant-1 e gli altri decision model ripetono la configurazione migliore.

Regole

1. Una modifica per prova, con il suo `configs/tok_<id>.json` (opzioni della testa, della loss, della decodifica).
2. Si decide su `dev_evid_select` guardando: frasi gold colpite, insieme completo, F1 sui caratteri, frasi colpite con evidenza a ≥ 10 frasi di distanza; più i negativi accesi su `dev_evid_neg`.

3. **Una modifica si tiene** se aumenta le frasi colpite di almeno 1 punto o l'insieme completo di almeno 2, senza far scendere le altre metriche di più di 1 punto, e restando nel limite di latenza.
4. A parità di risultato, la versione più semplice.
5. Guadagni sotto 2 punti si ripetono con 2 semi in più prima di tenerli.

La testa di partenza: H0

Un layer (31), niente attenzione, niente contesto locale né globale: il punteggio del token viene dal suo vettore e da `q` (`use_att=False, use_local=False, use_global=False`). Loss completa, decodifica di default.

Prima domanda: H0 batte E-LEX ed E-ATT? Se no, prima di provare altro si controllano `tok_char` ed etichette (06 §6).

Esperimenti principali (in quest'ordine)

Nome	Che cosa si aggiunge	Che cosa si guarda
H1	contesto locale (3 convoluzioni)	tratti meno frammentati, F1 sui caratteri
H2	attenzione della posizione di risposta per token	frasi colpite, soprattutto quelle "ponte"
H3	contesto globale (transformer sui blocchi)	evidenza lontana, insieme completo
H4	soglia del documento `s0` contro una soglia fissa scelta sul dev	insieme completo, numero di tratti
H5	termini della loss, uno alla volta: senza `L_hit`; senza `L_out`; senza `L_tv`	quanto serve ciascuno
H6	4 layer (19, 23, 27, 31) invece di 2	frasi colpite; spazio su disco doppio
H7	blocchi da 8 e da 32 token invece di 16	evidenza lontana, latenza

Lettura di H3: se il contesto globale non migliora l'evidenza lontana, o le parti lontane non si vedono (blocchi troppo grandi?) o i dati hanno poca evidenza lontana; HoVer ne ha molta (6.608 positivi con frasi gold a ≥ 20 frasi), quindi ci si aspetta un guadagno netto.

Controlli sulla configurazione migliore

Nome	Che cosa si cambia	Che cosa dice
H8	vettori estratti senza preambolo	quanto vale il preambolo; si ripete su elephant-1, che è bidirezionale
H9	fase 1b: vettori completi e riduzione appresa invece della PCA	se la PCA perde informazione
H10	training con le righe generate a intervalli esatti (passo 3)	se la testa impara tratti più stretti di una frase
H11	ω = 0,3 / 0,5 / 1	gestione delle annotazioni incomplete
H12	peso delle righe HoVer `db_split`: 0 / 0,5 / 1	se le righe recuperate dal db aiutano o fanno rumore
H13	α (peso dei token gold in `L_tok`) = 1 / 3 / 10	quanto spingere ad accendere frasi intere

Testa per token contro testa per frase

Nome	Che cosa
T-vs-F	stessa fase (1a), stessi dati, stesso modello: questa testa contro una **testa per frase** (un punteggio per frase del divisore NLTK, dallo stato dell'ultimo token e dalla media dei token di ogni frase, con un transformer sulle frasi), confrontate con le metriche per frase del passo 10 §3 e su `test_nopunct`

Lettura: se la testa per token resta entro ~1 punto da quella per frase sulle metriche per frase, ed è nettamente meglio senza punteggiatura, si adotta la testa per token (nessun divisore in produzione). Se è nettamente sotto, il prezzo dell'indipendenza dal divisore è troppo alto: si torna a una testa per frase con il divisore incluso nella repo.

elephant-1 e gli altri modelli

Nome	Che cosa
E0	elephant-1: PCA, estrazione, configurazione migliore di KnowLine
X0	gli altri decision model: prima quelli con il codice degli autori in transformers (Jeffan-4B, d3-flash, OneJev, Nimble), poi StartLux (maschera da ricostruire), pplx e Clef (testa propria)
E2	fase 2, solo se su KnowLine ha dato un guadagno chiaro e il modello è sotto gli obiettivi
L1	latenza per ogni modello rilasciato

Controlli finali

Nome	Che cosa
C1	3 semi della testa finale per modello: media e variabilità
C2	sufficienza e necessità su tutti i set di test
C3	testa contro E-ABL: quanto ci si avvicina con un forward a un metodo da centinaia di forward
C4	confronto fra i modelli sulle righe giudicate supportate da tutti

Registro

`data/evidence_tok/runs/registry.md`, una riga per prova (scritta in automatico dallo script di training, tranne la decisione):

Nome	Modello	Fase	Modifica	Frasi colpite	Insieme completo	F1 caratteri	Colpite ≥ 10 frasi	Tratti per riga	Negativi accesi	Sufficienza	Δ latenza	Decisione
E-LEX	—	—	—								—	riferimento

Nome	Modello	Fase	Modifica	Frasi colpite	Insieme completo	F1 caratteri	Colpite ≥ 10 frasi	Tratti per riga	Negativi accesi	Sufficienza	Δ latenza	Decisione
H0	KnowLine-4B	—	—									

Passo 12 — Server e rilascio

In breve

Un server (`evidence/serve_tokens.py`) che per ogni richiesta fa **un forward** e restituisce verdetto e tratti di evidenza. **Nessun divisore in frasi**: il documento entra così com'è e l'uscita sono intervalli di caratteri. Usa lo stesso codice di rendering (06), hook (01), testa e decodifica (08) usato in valutazione: quello che si è misurato è quello che gira.

1. Che cosa fa il server per ogni richiesta

```
richiesta {document, claim, [display: "spans" | "sentences"]}
  1. prompt con il preambolo (codice degli autori del modello) → input_ids; token del documento e loro caratteri
 ↪  (tok_char)
  2. UN forward con gli hook: punteggi delle opzioni, vettori dei token del documento proiettati, attenzione, q
  3. P(supportato); se non supera la soglia → risposta senza evidenza
  4. testa → s[t], s0 → decodifica → tratti in caratteri sul documento originale
  5. solo se display = "sentences": ogni tratto si allarga alla frase che lo contiene, con il divisore del server
```

```python
def answer(document, claim, display="spans"):
    ex = render_tokens(document, claim, preamble=DECISION["preamble"])          # input_ids, answer_pos, doc_tok,
 ↪  tok_char
    logits, feats = model.forward_tokens(ex)                                     # un forward con gli hook
    p = p_supported(logits, DECISION["temperature"], DECISION["take"])
    if p <= DECISION["threshold"]:
        return {"p_supported": p, "supported": False, "evidence": []}
    s, s0 = head(**feats)
    spans = decode(s[0].tolist(), float(s0[0]), ex["tok_char"], document, **DECISION["decode"])
    if display == "sentences":
        spans = expand_to_sentences(spans, document)                             # solo visualizzazione
    return {"p_supported": p, "supported": True, "evidence": spans}
```

Il passo 5 è l'unico punto in cui può comparire un divisore, e serve solo a mostrare: cambiarlo (NLTK, regex, quello del client) non tocca la testa né i punteggi.

2. Come gira ogni modello

Il server ha bisogno degli **stati interni** del forward, non solo della risposta: il forward si fa in transformers, con il codice degli autori e gli hook.

Modello	Forward nel server
KnowLine-4B	front end degli autori (`knowline_server.py`) con backend transformers, più gli hook
elephant-1	motore `qev` in-process (legge la maschera a blocchi da `modeling_overrides`, solo da una cartella locale)
Jeffan-4B, d3-flash, pplx-decider-flash, Clef-Flash	il codice degli autori in transformers
OneJev1.6-9B, Nimble-9B-v3	transformers con il prompt degli autori: i loro runtime (Flide, motore Nimble) non espongono gli stati; il verdetto si verifica contro il loro runtime

Modello	Forward nel server
StartLux 9B/4B	transformers con la maschera bidirezionale ricostruita: il server degli autori è vLLM e non espone gli stati; il verdetto si verifica contro il loro server

Regole:

- servono `flash-linear-attention` e `causal-conv1d` (layer DeltaNet), altrimenti il forward è molto più lento;
- la proiezione dei vettori dei token si fa sulla GPU nell'hook: escono n × 256 numeri per layer;
- per KnowLine, nelle richieste lette due volte, si usano i token della seconda copia;
- richieste raggruppate per lunghezza simile; la testa è invariante al riempimento;
- documenti oltre il limite di token: errore esplicito (`document_too_long`), mai troncati.

3. La cartella di rilascio

`releases/<modello>-fc-evtok-<versione>/`:

File	Contenuto
pesi del verdetto	quelli del fine-tuning binario (o riaddestrati con il preambolo, o con la fase 2)
`token_head.safetensors`	pesi della testa (18 MB per KnowLine-4B, 19,6 MB per elephant-1)
`projection/`	media e proiezione per layer (PCA, o la riduzione appresa nella fase 1b)
`decision.json`	tutte le impostazioni (sotto)
`frozen.json`	hash di pesi, testa, proiezioni, `token_render.py`, `escape.py`, `decode.py`, `question.json`, split e mix
`report.md`	risultati dei test e latenza (10 §7)
`MODEL_CARD.md`	descrizione per chi usa il modello

```
{"model": "knowline-4b-fc-evtok-v1",
 "question_id": "…", "take": "…",
 "preamble": "Claim under review: {claim}",
 "temperature": 1.0, "threshold": 0.50,
 "evidence": {"layers": [23, 31], "attention_layers": [3, 7, 11, 15, 19, 23, 27, 31],
              "block": 16, "head_config": "configs/tok_<id>.json"},
 "decode": {"delta": 0.0, "max_gap": 3, "min_len": 4, "max_spans": 5, "force_one": true},
 "max_prompt_tokens": 32768}
```

(Valori di esempio.) Non c'è nessun campo per un divisore in frasi: il modello non ne ha bisogno. Se cambiano preambolo, layer o proiezione, la testa va riaddestrata.

4. Controlli prima del rilascio

Controllo	Deve dare
Verdetto del server = verdetto in valutazione (200 righe)	differenza media di P < 0,005
Tratti del server = tratti in valutazione (200 righe)	identici in almeno 199 righe
Richieste in batch con lunghezze diverse	stessi tratti che una alla volta
Documenti con ", \, a capo, <, emoji, testo non latino	`document[start:end]` è il testo atteso
Documento senza punteggiatura	risposta valida, tratti dentro il testo

Controllo	Deve dare
Latenza	entro il limite (10 §5)

5. Congelamento, test, model card

1. Scrivere `frozen.json`.
2. Eseguire **una sola volta** i set di test e la latenza; scrivere `report.md`.
3. Model card: verifica documento–claim in inglese; evidenzia **solo** quando il claim è supportato; non dice cosa manca né cosa contraddice; restituisce tratti di testo, non frasi; i tratti sono stati appresi da annotazioni a livello di frase, quindi tendono a coprire frasi intere o loro parti significative; prompt con una riga del claim prima del documento.
4. Licenze: quelle dei pesi di partenza (MIT per elephant-1; Apache 2.0 per KnowLine-4B, Jeffan-4B, d3-flash, OneJev, pplx-decider-flash, Clef-Flash; **CC BY-NC per StartLux e Nimble**, quindi derivati non commerciali). Dati della testa: HoVer CC BY-SA; SciFact e WiCE con le loro licenze.

Checklist e problemi frequenti

1. Checklist per passo

Passo 1 — Modelli e hook per token

- ☐ `flash-linear-attention` e `causal-conv1d` installati e importabili.
- ☐ `TokenTap` e `sdpa_capture` funzionanti per KnowLine-4B; test superati (verdetto, layer, attenzione contro "eager", maschera, documento giusto).
- ☐ PCA dei layer 23 e 31 di KnowLine-4B, con la varianza spiegata registrata.

Passo 2 — Dati

- ☐ `gold_char_sets` e `contra_char` aggiunti a `evidence/data/schema.py`, validati e testati.
- ☐ Dataset ricostruiti con `--hover-db-fallback`; 30 positivi per fonte letti con gli intervalli evidenziati.

Passo 3 — Dati generati (facoltativo per le prime prove)

- ☐ Articoli con posizioni esatte; ablazione con uscita in intervalli; evidenza spostata.

Passo 4 — Split e mix

- ☐ Split per `doc_id`; WiCE con `aggrefact_overlap` solo in test; dev e test di HoVer solo `exact`; `test_nopunct` costruito.
- ☐ Mix `ev_tok_head` con i vincoli rispettati.

Passo 5 — Preambolo

- ☐ Testo scelto; verdetto riusato o riaddestrato; `question.json` aggiornato.

Passo 6 — Dai caratteri ai token, estrazione

- ☐ `evidence/escape.py` con `escape_with_map` e `token_char_spans`; testato contro `json.dumps`.
- ☐ `tok_char` ricostruisce il documento (200 su 200); etichette dei token coprono le frasi gold.
- ☐ Vettori di KnowLine-4B salvati per il mix e i set dev, con `meta.json`.

Passo 7 — Baseline

- ☐ E-RAND, E-LEX, E-EMB, E-ATT su `dev_evid_select`; E-ABL su 300 righe per fonte.

Passo 8 — Testa, loss, decodifica

- ☐ `tests/test_token_head.py` passa (forme, invarianza al riempimento, esempio numerico, decodifica).

Passo 9 — Training

- ☐ Impara a memoria 64 righe; H0 riproducibile; parametri della decodifica scelti e confermati.

Passo 10 — Valutazione

☐ Metriche sulle frasi gold e sui caratteri; confronto con la testa per frase; `test_nopunct`; latenza A/B/C.

Passo 11 — Esperimenti

☐ H0–H7, controlli H8–H13, T-vs-F; registro compilato.

Passo 12 — Rilascio

☐ Controlli del server superati; `frozen.json` prima dei test; test una volta; model card.

2. Se succede…

Problema	Prima cosa da controllare	Poi
La testa non impara (risultati vicini al caso)	`tok_char` ed etichette dei token (passo 6, test)	`meta.json` dei vettori contro la configurazione
Evidenzia paragrafi interi	peso di `L_out`; buco massimo della decodifica	ω più alto; α più basso (H13)
Tratti frammentati, parole isolate	`L_tv`; contesto locale (H1)	lunghezza minima della decodifica
Colpisce una frase gold ma non la “ponte”	contesto globale (H3), attenzione (H2)	più evidenza spostata; blocchi più piccoli (H7)
Fa come E-LEX (accende le parole del claim)	negativi nel mix e loro peso	più negativi con parole in comune col claim
Accende tratti sui claim non supportati	peso di `mil_neg`	δ più alto nella decodifica
Precisione bassa ma sufficienza alta	leggere a mano i “falsi positivi”: spesso è evidenza valida non annotata	ω più basso per quella fonte
Peggio della testa per frase sulle metriche per frase	rumore delle etichette per token	righe generate a intervalli esatti (H10); peso di `L_hit` più alto
Spazio su disco insufficiente	numero di layer e di componenti PCA	1 layer; 128 componenti
Latenza oltre il limite	proiezione, convoluzioni, transformer misurati a parte	blocchi più grandi; meno convoluzioni
Tratti con i caratteri sbagliati	mappa dell’escaping	test con documenti con `"`, `\`, a capo
Training o server molto lenti	i kernel DeltaNet sono davvero usati?	righe raggruppate per lunghezza

Una strada da esplorare: etichette per token dal modello stesso

Il problema

Le etichette della testa per token sono grossolane: le fonti segnano frasi intere, quindi dentro una frase gold anche le parole irrilevanti hanno etichetta 1. La loss compensa (chiede di colpire ogni frase gold, non di accenderla tutta), ma la testa non riceve mai un segnale su **quale parte** della frase conta. Il vantaggio principale della testa per token, evidenziare solo ciò che serve, resta quindi poco sfruttato.

L’idea

Ricavare etichette più fini **dal decision model stesso**, offline, togliendo pezzi più piccoli di una frase e misurando quanto cambia il verdetto:

```
per ogni frase gold g di una riga positiva:
    si divide g in pezzi fissi di ~5 token (nessun divisore: solo posizioni)
    per ogni pezzo w:  importanza(w) = z(documento) − z(documento senza w)        z = margine del verdetto (sì −
↪   no)
```

I pezzi con importanza alta sono la parte della frase che regge il verdetto (il nome, la data, la relazione); quelli con importanza vicina a zero sono contorno. Le etichette della testa diventano morbide: 1 sui pezzi importanti, un valore più basso (0,3) sugli altri token della frase gold, 0 fuori.

Come farlo

1. **Righe**: i positivi di training con frasi gold (HoVer `exact`, SciFact, WiCE train).
2. **Calcolo** (`evidence/label_token_importance.py`), con KnowLine-4B: un forward per pezzo, solo dentro le frasi gold (in media 3 frasi × ~6 pezzi = ~18 forward per riga). KnowLine è causale: i prompt sono uguali fino al pezzo tolto, quindi si riusa il calcolo della parte iniziale comune.
3. **Prima verifica, senza training**: su 50 righe lette a mano, i pezzi con importanza più alta sono davvero la parte che dimostra il claim? Se no (il modello segue scorciatoie, o l'importanza è piatta), ci si ferma.
4. **Training** (esperimento H-IMP, fase 1a): `L_tok` usa le etichette morbide al posto di quelle per frase; `L_hit` e `L_out` restano uguali.
5. **Valutazione**: le metriche del passo 10, più la **lunghezza media dei tratti a parità di frasi gold colpite** (più corta = più precisa) e la sufficienza (i tratti più corti reggono ancora il verdetto?).

Quando tenerlo

Rispetto alla configurazione migliore senza H-IMP:

- frasi gold colpite e insieme completo non più di 1 punto sotto;
- tratti almeno del 30% più corti;
- sufficienza non più di 2 punti sotto.

Rischi

Rischio	Cosa fare
Togliere 5 token rende la frase sgrammaticata e il modello reagisce alla forma, non al contenuto	confrontare con la sostituzione dello stesso pezzo con "…"; tenere l'importanza solo dove le due misure concordano
Informazione ripetuta nella frase: importanza bassa per tutti i pezzi	le etichette morbide non scendono mai sotto 0,3 dentro la frase gold
Costo: ~18 forward per riga	solo frasi gold, riuso della parte comune del prompt, KnowLine-4B

Perché vale la pena

È il modo più diretto di usare ciò che la testa per token sa fare e una testa per frase no: mostrare la parte della frase che conta. Non servono nuove annotazioni, il lavoro è tutto offline e in produzione non cambia nulla.